

# Continual Learning on Noisy Data Streams via Self-Purified Replay

Chris Dongjoo Kim<sup>\*1,2</sup>, Jinseo Jeong<sup>\*2</sup>, Sangwoo Moon<sup>2</sup>, and Gunhee Kim<sup>2</sup>

<sup>1</sup>NALBI Inc.

<sup>2</sup>Department of Computer Science and Engineering, Seoul National University, South Korea  
{cdjkim, jinseo, sangwoo.moon}@vl.snu.ac.kr gunhee@snu.ac.kr

## Abstract

Continually learning in the real world must overcome many challenges, among which **noisy labels are a common and inevitable issue**. In this work, we present a replay-based continual learning framework that simultaneously addresses both catastrophic forgetting and noisy labels for the first time. Our solution is based on two observations; (i) **forgetting can be mitigated even with noisy labels via self-supervised learning**, and (ii) **the purity of the replay buffer is crucial**. Building on this regard, we propose two key components of our method: (i) a self-supervised replay technique named **Self-Replay which can circumvent erroneous training signals arising from noisy labeled data**, and (ii) the **Self-Centered filter that maintains a purified replay buffer via centrality-based stochastic graph ensembles**. The empirical results on MNIST, CIFAR-10, CIFAR-100, and WebVision with real-world noise demonstrate that our framework can maintain a highly pure replay buffer amidst noisy streamed data while greatly outperforming the combinations of the state-of-the-art continual learning and noisy label learning methods. The source code is available at <http://vision.snu.ac.kr/projects/SPR>

## 1. Introduction

The most natural form of input for an intelligent agent occurs sequentially. Hence, the ability to continually learn from sequential data has gained much attention in recent machine learning research. This problem is often coined as *continual learning*, for which three representative approaches have been proposed [60, 70, 20] including replay [53, 29, 69, 77, 74, 45], regularization [39, 97, 3], and expansion techniques [75, 94].

At the same time, learning from data riddled with noisy labels is an inevitable scenario that an intelligent agent must

overcome. There have been multiple lines of work to learn amidst noisy labels such as loss regularization [89, 102, 31], data re-weighting [72, 76], label cleaning [67, 43, 64], and training procedures [90, 36].

In this work, we aim to jointly tackle the problems of *continual learning* and *noisy label classification*, which to the best of our knowledge have not been studied in prior work. Noisy labels and continual learning are inevitable for real-world machine learning, as data comes in a stream possibly polluted with label inconsistency. Hence, the two are bound to intersect; we believe exploring this intersection may glean evidence for promising research directions and hopefully shed light on the development of sustainable real-world machine learning algorithms.

We take on the replay-based approach to tackle continual learning since it has often shown superior results in terms of performance and memory efficiency even with simplicity. Yet, we discover that **replaying a noisy buffer intensifies the forgetting process due to the fallacious mapping of previously attained knowledge**. Moreover, existing noisy label learning approaches show great limitations when coping within the online task-free setting [2, 68, 44, 37]. In their original forms, they assume that the whole dataset is given to purify the noise and thus are hampered by a small amount of data stored only in the replay buffer to either regularize, re-weight, or decide on its validity.

We begin by backtracking the root of the problem; if we naively store a sampled set of the noisy input stream into the replay buffer, it becomes riddled with noise, worsening the amount of forgetting. Thus, we discover the key to success is **maintaining a pure replay buffer**, which is the major motive of our novel framework named *Self-Purified Replay* (SPR). At the heart of our framework is **self-supervised learning** [16, 12, 30, 24], which allows to circumvent the erroneous training signals arising from the incorrect pairs of data and labels. Within the framework, we present our novel *Self-Replay* and *Self-Centered filter* that collectively cleanse noisy labeled data and continually learn from them. The

<sup>\*</sup>Equal Contribution

Self-Replay mitigates the noise intensified catastrophic forgetting, and the Self-Centered filter achieves a highly clean replay buffer even when restricted to a small portion of data at a time.

We outline the contributions of this work as follows.

1. To the best of our knowledge, this is the first work to tackle *noisy labeled continual learning*. We discover **noisy labels exacerbate catastrophic forgetting**, and it is critical to filter out such noise from the input data stream before storing them in the replay buffer.
2. We introduce a novel replay-based framework named Self-Purified Replay (SPR), for noisy labeled continual learning. SPR can not only maintain a clean replay buffer but also effectively mitigate catastrophic forgetting with a fixed parameter size.
3. We evaluate our approach on three synthetic noise benchmarks of MNIST [42], CIFAR-10 [41], CIFAR-100 [41] and one real noise dataset of WebVision [50]. Empirical results validate that SPR significantly outperforms many combinations of the state-of-the-art continual learning and noisy label learning methods.

## 2. Problem Statement

### 2.1. Noisy Labeled Continual Learning

We consider the problem of **online task-free continual learning** for classification where a sample  $\{x_t, y_t\}$  enters at each time step  $t$  in a non i.i.d manner **without task labels**. While previous works [69, 68, 44] assume  $\{x_t, y_t\}$  are correct (clean) samples, we allow the chance that a large portion of the data is falsely labeled.

### 2.2. Motivation: Noise induced Amnesia

We discover that if the data stream has noisy labels, it traumatically damages the continual learning model, analogous to *retrograde amnesia* [80], the **inability to recall experience of the past**. We perform some preliminary experiments on a sequential version of symmetric noisy MNIST and CIFAR-10 [56, 89] using experience replay with the conventional **reservoir sampling technique** [73, 100].

The empirical results in Figure 1 show that when trained with noisy labels, the model becomes much more prone to catastrophic forgetting [20, 60, 83, 70]. As the noise level increases from 0% to 60%, sharp decreases in accuracy are seen. Surprisingly, the dotted red circle in Figure 1(b) shows that in CIFAR-10 a fatally hastened forgetting occurs no matter the amount of noise.

We speculate that a critical issue that hinders the continual model is the corrupted replay buffer. An ideal replay buffer should shield the model from noisy labels altogether by being vigilant of all the incoming data for the maintenance of a clean buffer.

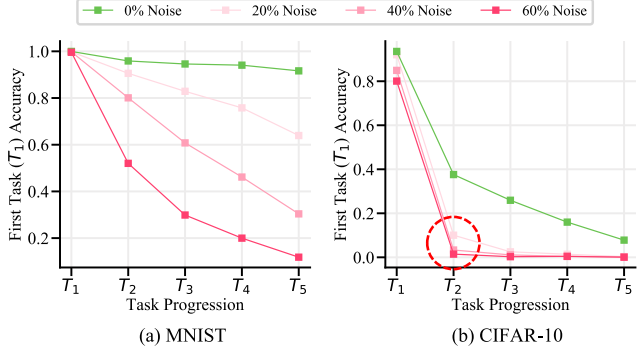


Figure 1. A noisy labeled continual learning on the symmetric noisy in (a) MNIST [42] and (b) CIFAR-10 [41] when using experience replay with the conventional reservoir sampling [100, 73]. At the end of each task, the accuracy of the first task ( $T_1$ ) is plotted. It shows that the noisy labels accelerate catastrophic forgetting. Notably, the dotted red circle in (b) indicates the significantly hastened forgetting process.

## 3. Approach to Noisy Labeled Continual Learning

We design an approach to continual learning with noisy labels by realizing the two interrelated subgoals as follows.

- G1. Reduce forgetting even with noisy labels:** The approach needs to mitigate catastrophic forgetting amidst learning from noisy labeled data.
- G2. Filter clean data:** The method should learn representations such that it identifies the noise as anomalies. Moreover, it should **enable this from a small amount of data since we do not have access to the entire dataset in online continual learning**.

Figure 2 overviews the proposed framework consisting of two buffers and two networks. The *delayed buffer*  $\mathcal{D}$  temporarily stocks the incoming data stream, and the *purified buffer*  $\mathcal{P}$  maintains the cleansed data. The *base* network addresses **G1** via self-supervised replay (Self-Replay) training (Section 3.1). The *expert* network is a key component of Self-Centered filter that tackles **G2** by obtaining **confidently clean samples via centrality** (Section 3.2). Both networks have the same architecture (*e.g.*, ResNet-18) with separate parameters.

Algorithm 1 outlines the training and filtering procedure. Whenever the delayed buffer  $\mathcal{D}$  is full, The Self-Centered filter powered by the expert network filters the clean samples from  $\mathcal{D}$  to the purified buffer  $\mathcal{P}$ . Then, the base network is trained via the self-supervision loss with the samples in  $\mathcal{D} \cup \mathcal{P}$ . The detail will be discussed in Section 3.1–3.2.

At any stage of learning, we **can perform downstream tasks (*i.e.*, classification) by duplicating the base network into the inference network, adding a final softmax layer, and**

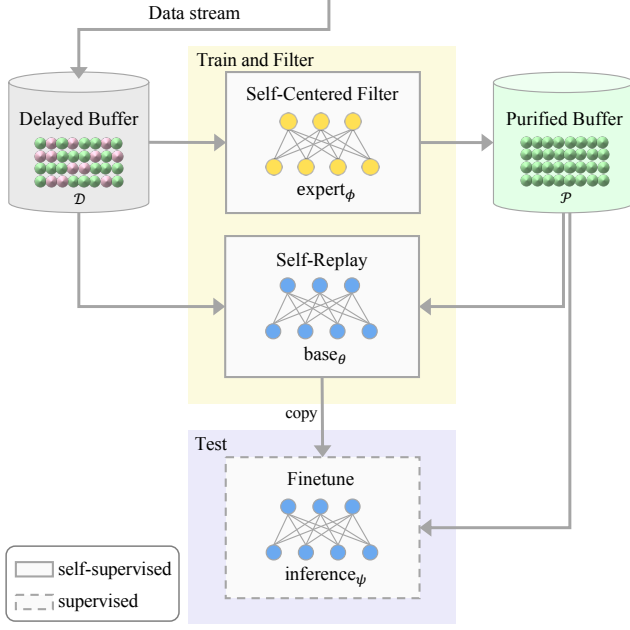


Figure 2. Illustration of the Self-Purified Replay (SPR) framework. We specify the training and filtering phase (in the yellow shade) in Algorithm 1, and the test phase (in the purple shade) in Algorithm 2.

finetuning it using the samples in  $\mathcal{P}$ . Algorithm 2 outlines this inference phase.

### 3.1. Self-Replay

Learning with noisy labeled data [67, 5, 57, 28] results in erroneous backpropagating signals when falsely paired  $x$  and  $y$  exist in the training set. Hence, we circumvent this error via learning only from  $x$  (without  $y$ ) using contrastive self-supervised learning techniques [7, 12, 30, 24]. That is, the framework first focuses on learning general representations via self-supervised learning from *all* incoming  $x$ . Subsequently, the downstream task (*i.e.*, supervised classification) finetunes the representation using only the samples in the purified buffer  $\mathcal{P}$ . Building on this concept in terms of continual learning leads to Self-Replay, which mitigates forgetting while learning general representations via self-supervised replay of the samples in the delayed and purified buffer ( $\mathcal{D} \cup \mathcal{P}$ ).

Specifically, we add a projection head  $g(\cdot)$  (*i.e.*, a one-layer MLP) on top of the average pooling layer of the base network, and train it using the normalized temperature-scaled cross-entropy loss [12]. For a minibatch from  $\mathcal{D}$  and  $\mathcal{P}$  with a batch size of  $B_d, B_p \in \mathbb{N}$  respectively, we apply random image transformations (*e.g.*, cropping, color jitter, horizontal flip) to create two correlated views of each sample, referred to as positives. Then, the loss is optimized to attract the features of the positives closer to each other while repelling them from the other samples in the batch, referred

#### Algorithm 1 Training and filtering phase of SPR

**Input:** Training data  $(x_t, y_t), \dots, (x_T, y_T)$  and initial parameters of base network  $\theta$ .

$\mathcal{D} = \mathcal{P} = \{\}$  // Initialize delayed and purified buffer

**for**  $t = 1$  **to**  $T$  **do**

**if**  $\mathcal{D}$  is full **then**

$\mathcal{P} \leftarrow \mathcal{P} \cup \text{Self-Centered Filter}(\mathcal{D})$  (section 3.2)

$\theta \leftarrow \text{Self-Replay using } \mathcal{D} \cup \mathcal{P}$  (section 3.1)

    reset  $\mathcal{D}$

**else**

    update  $\mathcal{D}$  with  $(x_t, y_t)$

**end if**

**end for**

#### Algorithm 2 Test phase of SPR

**Input:** Test data  $(x_t, y_t), \dots, (x_T, y_T)$ , parameters of the base network  $\theta$ , and purified buffer  $\mathcal{P}$

$\psi = \text{copy}(\theta)$  // Duplicate base model to inference model

$\psi \leftarrow \text{supervised finetune using } \mathcal{P}$

**for**  $t = 1$  **to**  $T$  **do**

  downstream classification for  $(x_t, y_t)$  using  $\psi$

**end for**

to as the negatives. The updated objective becomes

$$L_{self} = - \sum_{i=1}^{2(B_d+B_p)} \log \frac{e^{u_i^T u_j / \tau}}{\sum_{k=1}^{2(B_d+B_p)} \mathbb{1}_{k \neq i} e^{u_i^T u_k / \tau}}. \quad (1)$$

We denote  $(x_i, x_j)$  as the positives and  $x_k$  as the negatives.  $u_i = \frac{g(x_i)}{\|g(x_i)\|_2}$  is the  $\ell_2$  normalized feature, and  $\tau > 0$  is the temperature. Every time when the delayed buffer is full, we train the base network with this loss.

**Empirical supports.** Figure 3 shows some empirical results about the validity of Self-Replay for noisy labeled continual learning.

- Figure 3(a) shows a quantitative examination on downstream classification tasks. It indicates that self-supervised learning leads to a better representation, and eventually outperforms the supervised one by noticeable margins.
- Figure 3(b) exemplifies the superiority of Self-Replay in continual learning. We contrast the performances of continually trained Self-Replay (as proposed) against intermittently trained Self-Replay, which trains *offline* with only the samples in the purified buffer at the end of each task. The colored areas in Figure 3(b) indicate how much the continually learned representations alleviate the forgetting and benefit the knowledge transfers among the past and future tasks.

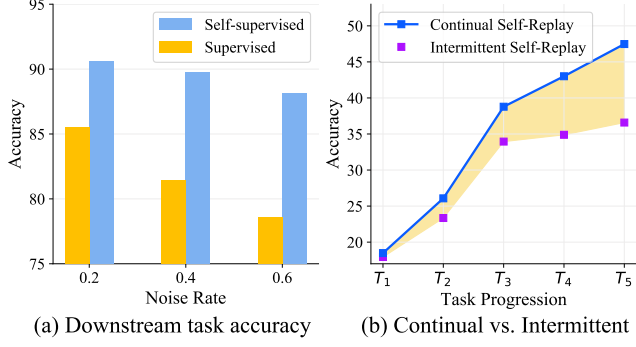


Figure 3. **Empirical support for Self-Replay** with ResNet18 as the base network on CIFAR-10. (a) Comparison of overall accuracy of the finetuned downstream classification between self-supervised and supervised representations trained on various noise rates. The self-supervised indicates that the base network trained using only  $x$  as proposed, while the supervised means training with possibly noisy  $(x, y)$  pairs. (b) The benefits of continual Self-Replay over the intermittent Self-Replay by comparing the test set accuracy of finetuned models. The intermittent Self-Replay means training only with contents of the purified buffer up to and including the current task.

### 3.2. Self-Centered Filter

The goal of the Self-Centered filter is to obtain confidently clean samples; specifically, it assigns the probability of being clean to all the samples in the delayed buffer.

**Expert Network.** The expert network is prepared to *feature* the samples in the delayed buffer. These features are used to compute the centrality of the samples, which is the yardstick of selecting clean samples. Inspired by the success of self-supervised learning good representations in Self-Replay, the expert network is also trained with the self-supervision loss in Eq. 12 with only difference that we use the samples in  $\mathcal{D}$  only (instead of  $\mathcal{D} \cup \mathcal{P}$  for the base network).

**Centrality.** At the core of the Self-Centered filter lies centrality [62], which is rooted in graph theory to identify the most influential vertices within a graph. We use a variant of the *eigenvector centrality* [6], which is grounded on the concept that a link to a highly influential vertex contributes to centrality more than a link to a lesser influential vertex.

First, weighted undirected graphs  $G := (V, E)$  are constructed *per unique class label* in the delayed buffer. We assume that the clean samples form the largest clusters in the graph of each class. Each vertex  $v \in V$  is a sample of the class, and the edge  $e \in E$  is weighted by the cosine similarity between the features from the expert network. For the adjacency matrix  $\mathbf{A} = (a_{v,u})_{|V| \times |V|}$ . Then the eigenvector

centrality is formulated as

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u = \frac{1}{\lambda} \sum_{u \in V} a_{v,u} c_u, \quad (2)$$

where  $N(v)$  is the neighboring set of  $v$ ,  $\lambda$  is a constant and  $a_{v,u}$  is the truncated similarity value within  $(0, 1]$ . Eq. 2 can be rewritten in vector notation as  $\mathbf{A}\mathbf{c} = \lambda\mathbf{c}$ , where  $\mathbf{c}$  is a vectorized centrality over  $V$ . The principal eigenvector  $\mathbf{c}$  can be computed by the power method [87], and it corresponds to the eigenvector centrality for the vertices in  $V$ .

**Beta Mixture Models.** The centrality quantifies which samples are the most influential (or the cleanest) within the data of identical class labels. However, the identically labeled data contains both *clean* and *noisy* labeled samples, in which the noisy ones may deceptively manipulate the centrality score, leading to an indistinct division of the clean and noisy samples' centrality scores. Hence, we compute the probability of cleanliness per sample via fitting a Beta mixture model (BMM) [33] to the centrality scores as

$$p(c) = \sum_{z=1}^Z \pi_z p(c|z), \quad (3)$$

where  $c > 0$  is the centrality score,  $\pi_z$  is the mixing coefficients, and  $Z \in \mathbb{N}$  is the number of components. Beta distribution for  $p(c|z)$  is a suitable choice due to the skewed nature of the centrality scores. We set  $Z = 2$ , indicating the clean and noisy components, and it is empirically the best in terms of accuracy and computation cost. We use the EM algorithm [15] to fit the BMM through which we obtain the posterior probability

$$p(z|c) = \frac{\pi_z p(c|\alpha_z, \beta_z)}{\sum_{j=1}^Z \pi_j p(c|\alpha_j, \beta_j)}, \quad (4)$$

where  $\alpha_z, \beta_z > 0$  are the latent distribution parameters. Please refer to the appendix for details of computing  $p(z|c)$ .

Among the  $Z = 2$  components, we can easily identify the *clean* component as the one that has the higher  $c$  scores (*i.e.*, a larger cluster). Then, the *clean posterior*  $p(z = \text{clean}|c)$  defines the probability that centrality  $c$  belongs to the clean component, which is used as the probability to enter and exit the purified buffer,  $\mathcal{P}$ . After the selected samples enter our full purified buffer, the examples with the lowest  $p(z = \text{clean}|c)$  are sampled out accordingly.

#### 3.2.1 Stochastic Ensemble

Since our goal is to obtain the *most* clean samples as possible, we want to further sort out the possibly noisy samples. We achieve this by introducing a *stochastic ensemble* of BMMs, enabling a more noise robust posterior than the

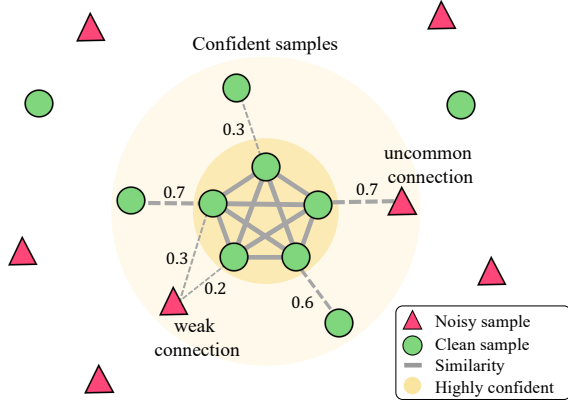


Figure 4. Illustration of graph manipulation via Stochastic Ensemble, which severs weak and uncommon connections and probabilistically focus on confident and clean data within the graph.

non-stochastic posterior  $p(z = \text{clean}|c)$  in the previous section.

First, we prepare for stochastic ensembling by sampling multiple *binary* adjacency matrices  $\{\mathbf{A}\}$  from a Bernoulli distribution over  $\mathbf{A}$ . For each class  $l$ , we impose a conditional Bernoulli distribution over  $\mathbf{A}$  as

$$p(\mathbf{A}|D_l) = \prod_{d_i, d_j \in D_l} \text{Bern} \left( \mathbf{A}_{ij} | \text{ReLU} \left( \frac{d_i \cdot d_j}{||d_i|| ||d_j||} \right) \right), \quad (5)$$

where  $D_l$  is the set of penultimate feature of class  $l$  from the expert network. We find that it is empirically helpful to truncate the dissimilar values to 0 (ReLU) and use the cosine similarity value as the probability. We replace the zeros in  $\mathbf{A}$  with a small positive value to satisfy the requirement of Perron-Frobenius theorem<sup>1</sup>. Then, our reformulated robust posterior probability is

$$p(z|D_l) \propto \int_{\mathbf{A}} p(z|\text{cent}(\mathbf{A})) dp(\mathbf{A}|D_l), \quad (6)$$

where  $\text{cent}(\cdot)$  is the centrality scores from Eq. 2, and  $p(z|\text{cent}(\mathbf{A}))$  can be obtained in the same manner as the *non-stochastic* posterior in the previous section. We approximate the integral using Monte Carlo sampling for which we use  $E_{max}$  as the sample size. Essentially, we fit the mixture models on different stochastic graphs to probabilistically carve out more confidently noisy samples by retaining the strong and dense connections while severing weak or uncommon connections. This is conceptually illustrated in Figure 7.

<sup>1</sup>Perron-Frobenius theorem states when  $\mathbf{A}$  has positive entries, it has a unique largest real eigenvalue, whose corresponding eigenvector have strictly positive components.

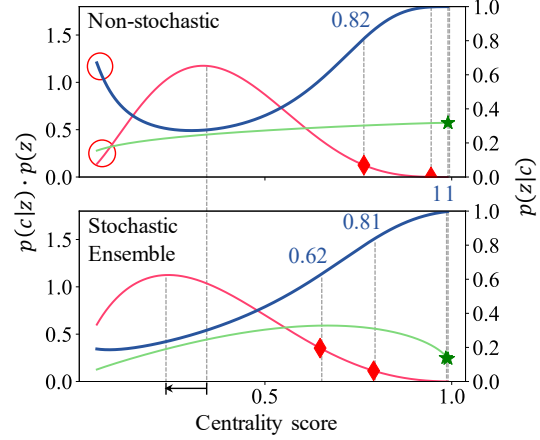


Figure 5. Comparison of non-stochastic and Stochastic Ensemble on CIFAR-10 with 40% noise. Stochastic Ensemble produces more confidently clean samples by shifting  $p(c|z = \text{noisy}) \cdot p(z = \text{noisy})$  to the left, and suppressing the cases where  $p(c|z = \text{noisy}) \cdot p(z = \text{noisy})$  dips below  $p(c|z = \text{clean}) \cdot p(z = \text{clean})$ .

Figure 5 shows some empirical evidence where the stochastic ensemble addresses the two issues to achieve a noise robust posterior  $p(z|D_l)$ .

#### Empirical Supports.

Figure 5 shows some empirical evidence where the stochastic ensemble addresses the two issues to achieve a noise robust posterior  $p(z|D_l)$ .

- First, a small portion of noisy samples are falsely confident and are consequently assigned a high centrality score. Stochastic ensembling is able to suppress these noisy samples, as indicated in Figure 5, where the mode of  $p(c|z = \text{noisy}) \cdot p(z = \text{noisy})$  (red curve) is shifted to the left by a noticeable margin.
- Second, there are some cases where  $p(c|z = \text{noisy}) \cdot p(z = \text{noisy})$  drops below the  $p(c|z = \text{clean}) \cdot p(z = \text{clean})$  leading to a high  $p(z = \text{clean}|c)$  for the noisy instances, indicated with red circles in Figure 5. The stochastic ensemble of differing  $\mathbf{A}$ s can mitigate such problematic cases to drown out the unexpected noise.

## 4. Related Works

### 4.1. Continual Learning

There have been three main branches to train a model from continual data streams: regularization [52, 19, 39, 3], expansion [75, 94, 44], and replay [53, 9, 10, 73, 34]. Replay-based approaches maintain a fixed-sized memory to rehearse back to the model to mitigate forgetting. Several works [53, 9, 10] reserve the space for data samples of previous tasks, while others [77] uses a generative model. Some works [73, 34] combine rehearsal with meta-



learning to find the balance between transfer and interference. We defer more comprehensive survey including all three branches of continual learning to the appendix.

**Online Sequential Learning.** In the online sequential learning scenario, a model can only observe the training samples once. Hence, many works propose methods for maintaining the buffer [29, 69, 37] or selecting the samples to be rehearsed [2]. Recently, [82] adopts graphs to represent relational structures between samples, and [25] employs the meta-loss for learning per-parameter learning rates along with model parameters.

Akin to our work, Gdumb [68] and MBPA++ [14] also train the model at inference time. However, greedily selecting samples to be reserved inevitably leads to degradation from noisy labeled data. Furthermore, discarding the samples that cannot enter the buffer as done in Gdumb may lead to information loss since it only relies on the buffer as its source of training.

## 4.2. Noisy Labels

Learning with noisy labeled data has long been studied [98, 5, 57, 35]. Several works design the noise corrected losses [91, 28, 47, 4, 89] so that the loss minimization of the whole data becomes similar to that of clean samples. Other works propose to use a noise transition matrix to correct the loss [66, 23, 31, 102]. There have been approaches that aim to suppress the contribution of noisy samples by re-weighting the loss [88, 72]. Techniques that repair labels [40, 85, 51, 79, 27, 59] or directly learn them [81, 93] are also viable options for learning from noisy labeled data. Recently, filtering methods based on training dynamics [32, 67, 61] have gained much popularity, based on the observation that models tend to learn clean data first and memorize the noisy labeled data later. Small loss sample selection [36, 76, 46] techniques by co-teaching [90, 21, 26, 95, 58, 11] identify noisy samples with multiple models in the same vein. Some works use graphs for offline learning from a large-scale noisy dataset [101, 99]. On the other hand, we use a small dataset in the delayed buffer from an online data stream without ground-truth labels; instead we adopt self-supervision to obtain features for the Self-Centered filter.

None of the works mentioned above address continual learning from noisy labeled data streams. Although [59, 48] also use self-supervised learning with noisy labeled data, they focus on the loss or prediction from the model for selecting suspicious samples. In the experiments on Table 6, we will show that training dynamics-based filtering techniques are not viable in noisy labeled continual learning. On the other hand, we provide the algorithm that identifies the clean samples while learning from a purified buffer in an online manner.

## 4.3. Self-supervised learning

Self-supervised learning is currently receiving an enormous amount of attention in machine learning research. The pretext task that trains a model by predicting hidden information within the data includes patch orderings [17, 63], image inpainting [65], colorization [92], and rotations [22, 13], to name a few. There also have been works that utilize the contrastive loss [12, 30, 49]; especially, SimCLR [12] proposes a simplified contrastive learning method, which enables representation learning by pulling the randomly transformed samples from the same image closer while pushing ones apart from other images within the batch. Recently, this instance-wise contrastive learning is extended to prototypical contrastive learning [49] to encode the semantic structures within the data.

## 5. Experiments

In our evaluation, we compare SPR with other state-of-the-art models in the online task-free continual learning scenario with label noise. We test on three benchmark datasets of MNIST [42], CIFAR-10 [41] and CIFAR-100 [41] with symmetric and asymmetric random noise, and one large-scale dataset of WebVision [50] with real-world noise on the Web. We also empirically analyze Self-Replay and the Self-Centered filter from many aspects.

### 5.1. Experimental Design

We explicitly ground our experiment setting based on the recent suggestions for robust evaluation in continual learning [1, 18, 84] as follows. (i) *Cross-task resemblance*: Consecutive tasks in MNIST [42], CIFAR-10 [41], CIFAR-100 [41], WebVision [50] are partly correlated to contain neighboring domain concepts. (ii) *Shared output heads*: A single output vector is used for all tasks. (iii) *No test-time task labels*: Our approach does not require explicit task labels during both training and test phase, often coined as *task-free continual learning* in [69, 44, 37]. (iv) *More than two tasks*: MNIST [42], CIFAR-10 [41], CIFAR-100 [41] and WebVision [50] contain five, five, twenty, and seven tasks, respectively.

We create a synthetic noisy labeled dataset from MNIST and CIFAR-10 using two methods. First, the *symmetric* label noise assigns  $\{20\%, 40\%, 60\%\}$  samples of the dataset to other labels within the dataset by a uniform probability. We then create five tasks by selecting random class pairs without replacement. Second, the *asymmetric* label noise attempts to mimic the real-world label noise by assigning other similar class labels (e.g.,  $5 \leftrightarrow 6$ , cat  $\leftrightarrow$  dog). We use the similar classes chosen in [66] to contaminate  $\{20\%, 40\%\}$  samples of the dataset with similar class pairs. Each task consists of the samples from each corrupted class pair. CIFAR-100 has 20 tasks where the *random symmetric* set-

| noise rate (%)         | MNIST       |             |             |             |             | CIFAR-10    |             |             |             |             | WebVision   |
|------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
|                        | symmetric   |             |             | asymmetric  |             | symmetric   |             |             | asymmetric  |             | real noise  |
|                        | 20          | 40          | 60          | 20          | 40          | 20          | 40          | 60          | 20          | 40          | unknown     |
| Multitask 0% noise [8] |             |             | 98.6        |             |             |             |             | 84.7        |             |             | -           |
| Multitask [8]          | 94.5        | 90.5        | 79.8        | 93.4        | 81.1        | 65.6        | 46.7        | 30.0        | 77.0        | 68.7        | 55.5        |
| Finetune               | 19.3        | 19.0        | 18.7        | 21.1        | 21.1        | 18.5        | 18.1        | 17.0        | 15.3        | 12.4        | 11.9        |
| EWC [39]               | 19.2        | 19.2        | 19.0        | 21.6        | 21.1        | 18.4        | 17.9        | 15.7        | 13.9        | 11.0        | 10.0        |
| CRS [86]               | 58.6        | 41.8        | 27.2        | 72.3        | 64.2        | 19.6        | 18.5        | 16.8        | 28.9        | 25.2        | 19.3        |
| CRS + L2R [72]         | 80.6        | 72.9        | 60.3        | 83.8        | 77.5        | 29.3        | 22.7        | 16.5        | 39.2        | 35.2        | -           |
| CRS + Pencil [93]      | 67.4        | 46.0        | 23.6        | 72.4        | 66.6        | 23.0        | 19.3        | 17.5        | 36.2        | 29.7        | 26.6        |
| CRS + SL [89]          | 69.0        | 54.0        | 30.9        | 72.4        | 64.7        | 20.0        | 18.8        | 17.5        | 32.4        | 26.4        | 21.5        |
| CRS + JoCoR [90]       | 58.9        | 42.1        | 30.2        | 73.0        | 63.2        | 19.4        | 18.6        | 21.1        | 30.2        | 25.1        | 19.5        |
| PRS [37]               | 55.5        | 40.2        | 28.5        | 71.5        | 65.6        | 19.1        | 18.5        | 16.7        | 25.6        | 21.6        | 19.0        |
| PRS + L2R [72]         | 79.4        | 67.2        | 52.8        | 82.0        | 77.8        | 30.1        | 21.9        | 16.2        | 35.9        | 32.6        | -           |
| PRS + Pencil [93]      | 62.2        | 33.2        | 21.0        | 68.6        | 61.9        | 19.8        | 18.3        | 17.6        | 29.0        | 26.7        | 26.5        |
| PRS + SL [89]          | 66.7        | 45.9        | 29.8        | 73.4        | 63.3        | 20.1        | 18.8        | 17.0        | 29.6        | 24.0        | 21.7        |
| PRS + JoCoR [90]       | 56.0        | 38.5        | 27.2        | 72.7        | 65.5        | 19.9        | 18.6        | 16.9        | 28.4        | 21.9        | 20.2        |
| MIR [2]                | 57.9        | 45.6        | 30.9        | 73.1        | 65.7        | 19.6        | 18.6        | 16.4        | 26.4        | 22.1        | 17.2        |
| MIR + L2R [72]         | 78.1        | 69.7        | 49.3        | 79.4        | 73.4        | 28.2        | 20.0        | 15.6        | 35.1        | 34.2        | -           |
| MIR + Pencil [93]      | 70.7        | 34.3        | 19.8        | 79.0        | 58.6        | 22.9        | 20.4        | 17.7        | 35.0        | 30.8        | 22.3        |
| MIR + SL [89]          | 67.3        | 55.5        | 38.5        | 74.3        | 66.5        | 20.7        | 19.0        | 16.8        | 28.1        | 22.9        | 20.6        |
| MIR + JoCoR [90]       | 60.5        | 45.0        | 32.8        | 72.6        | 64.2        | 19.6        | 18.4        | 17.0        | 27.6        | 23.5        | 19.0        |
| GDumb [68]             | 70.0        | 51.5        | 36.0        | 78.3        | 71.7        | 29.2        | 22.0        | 16.2        | 33.0        | 32.5        | 30.4        |
| GDumb + L2R [72]       | 65.2        | 57.7        | 42.3        | 67.0        | 62.3        | 28.2        | 25.5        | 18.8        | 30.5        | 30.4        | -           |
| GDumb + Pencil [93]    | 68.3        | 51.6        | 36.7        | 78.2        | 70.0        | 26.9        | 22.3        | 16.5        | 32.5        | 29.7        | 26.9        |
| GDumb + SL [89]        | 66.7        | 48.6        | 27.7        | 73.4        | 68.1        | 28.1        | 21.4        | 16.3        | 32.7        | 31.8        | 30.8        |
| GDumb + JoCoR [90]     | 70.1        | 56.9        | 37.4        | 77.8        | 70.8        | 26.3        | 20.9        | 15.0        | 33.1        | 32.2        | 24.2        |
| Self-Centered filter   | 80.1        | 79.0        | 77.4        | 80.0        | 79.6        | 36.5        | 35.7        | 32.5        | 37.1        | 36.9        | 33.0        |
| Self-Replay            | 81.5        | 69.2        | 43.0        | 86.3        | 78.9        | 40.1        | 31.4        | 22.4        | 44.1        | 43.2        | <b>48.0</b> |
| SPR                    | <b>85.4</b> | <b>86.7</b> | <b>84.8</b> | <b>86.8</b> | <b>86.0</b> | <b>43.9</b> | <b>43.0</b> | <b>40.0</b> | <b>44.5</b> | <b>43.9</b> | 40.0        |

Table 1. **Overall accuracy** of noisy labeled continual learning after all sequences of tasks are trained. The buffer size is set to 300, 500, 1000 for MNIST, CIFAR-10 and WebVision, respectively. Some empty slots on WebVision are due to the unavailability of clean samples required by L2R for training [72]. The results are the mean of five unique random seed experiments. We report best performing baselines on different episodes with variances in the appendix.

ting has 5 random classes per task with uniform noise across 100 classes. The *superclass symmetric* setting uses each superclass [41, 44] containing 5 classes as a task where the noise is randomized only within the classes in the superclass. In WebVision, we use the top 14 largest classes in terms of the data size, resulting in 47,784 images in total. We curate seven tasks with randomly paired classes.

We fix the delayed buffer and the replay (purified) buffer size to 300, 500, 1000, 5000 for MNIST, CIFAR-10, WebVision, and CIFAR-100, respectively. The purified buffer maintains balanced classes as in [37, 68]. We fix the stochastic ensemble size,  $E_{max} = 5$  unless stated otherwise. For the base model, we use an MLP with two hidden layers for all MNIST experiments and ResNet-18 for CIFAR-10, CIFAR-100, and WebVision experiments. Please refer to the appendix for experiment details.

## 5.2. Baselines

Since we opt for continual learning from noisy labeled data streams, we design the baselines combining existing state-of-the-art methods from the two domains of continual learning and noisy label learning.

We explore the replay-based approaches that can learn in the online task-free setting. We thus choose (i) Conventional Reservoir Sampling (CRS) [73], (ii) Maximally Interfered Retrieval (MIR) [2], (iii) Partitioning Reservoir Sampling (PRS) [37] and (iv) GDumb [68].

For noisy label learning, we select six models to cover many branches of noisy labeled classification. They include (i) SL loss correction [89], (ii) semi-supervised JoCoR [90], (iii) sample reweighting L2R [72], (iv) label repairing Pencil [93], (v) training dynamic based detection AUM [67] and (vi) cross-validation based INCV [11].

| noise rate (%)      | random symmetric |             |             | superclass symmetric |             |             |
|---------------------|------------------|-------------|-------------|----------------------|-------------|-------------|
|                     | 20               | 40          | 60          | 20                   | 40          | 60          |
| GDumb + L2R [72]    | 15.7             | 11.3        | 9.1         | 16.3                 | 12.1        | 10.9        |
| GDumb + Pencil [93] | 16.7             | 12.5        | 4.1         | 17.5                 | 11.6        | 6.8         |
| GDumb + SL [89]     | 19.3             | 13.8        | 8.8         | 18.6                 | 13.9        | 9.4         |
| GDumb + JoCoR [90]  | 16.1             | 8.9         | 6.1         | 15.0                 | 9.5         | 5.9         |
| SPR                 | <b>21.5</b>      | <b>21.1</b> | <b>18.1</b> | <b>20.5</b>          | <b>19.8</b> | <b>16.5</b> |

Table 2. **CIFAR100 results** of noisy labeled continual learning after all sequences of tasks are trained. The results are the mean of five unique random seed experiments.

| noise rate (%) | MNIST       |             |             |            |             |             | CIFAR-10    |             |             |             |    |    |
|----------------|-------------|-------------|-------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|----|----|
|                | symmetric   |             |             | asymmetric |             |             | symmetric   |             |             | asymmetric  |    |    |
|                | 20          | 40          | 60          | 20         | 40          | 60          | 20          | 40          | 60          | 20          | 40 | 60 |
| AUM [67]       | 7.0         | 16.0        | 11.7        | 30.0       | 29.5        | 36.0        | 24.0        | 11.7        | 46.0        | 30.0        |    |    |
| INCV [11]      | 23.0        | 22.5        | 14.3        | 37.0       | 31.5        | 22.0        | 18.5        | 9.3         | 37.0        | 30.0        |    |    |
| Non-stochastic | 79.5        | 96.3        | 84.5        | 96.0       | 88.5        | 50.5        | 54.5        | 38.0        | 53.0        | 50.5        |    |    |
| SPR            | <b>96.0</b> | <b>96.5</b> | <b>93.0</b> | <b>100</b> | <b>96.5</b> | <b>75.5</b> | <b>70.5</b> | <b>54.3</b> | <b>69.0</b> | <b>60.0</b> |    |    |

Table 3. **Filtered noisy label percentage** in the purified buffer (e.g., out of 20% symmetric noise, SPR filters 96% of noise). We compare SPR with  $E_{max} = 5$  to two other state-of-the-art label filtering methods.

### 5.3. Results

**Overall performance.** Table 1 compares the noisy labeled continual learning performance (classification accuracy) between our SPR and baselines on MNIST, CIFAR-10 and WebVision. Additionally, Table 2 compares SPR against the best performing baselines on CIFAR-100 with random symmetric noise and superclass symmetric noise. SPR performs the best in all symmetric and asymmetric noise types with different levels of 20%, 40%, and 60% as well as real noise. Multitask is an upper-bound trained with an optimal setting with perfectly clean data (i.e., the 0% noise rate) and offline training. Finetune is reported as a lower-bound performance since it performs online training with no continual or noisy label learning technique.

Notably, SPR works much better than L2R [72], which additionally uses 1000 clean samples for training, giving it a substantial advantage over all the other baselines. SPR also proves to be much more effective than GDumb [68], which is the most related method to ours, even when combined with different noisy label learning techniques.

Moreover, the addition of state-of-the-art noisy label techniques is not always beneficial. This may be because existing noisy label techniques usually assume a large dataset, which is required to reliably estimate the training dynamics to mitigate the noise by regularizing, repairing, and or filtering. However, the online learning setting is limited by a much smaller dataset (i.e., in the purified buffer), leading to a difficult training of the noisy label techniques.

**Ablation Study.** To study the effectiveness of each com-

ponent, two variants of our model that only use Self-Replay or the Self-Centered filter is tested. That is, the Self-Replay variant does not use any cleaning methods (i.e., use conventional reservoir sampling to maintain the purified buffer). The Self-Centered filter variant finetunes a randomly initialized inference network on the purified buffer instead of finetuning it on the duplicate of the base network. Both variants outperform all the baselines (excluding L2R) in all three datasets, and combining them our model performs the best on MNIST and CIFAR-10 with all noise levels. However, WebVision is the only dataset where no synergetic effect is shown, leaving Self-Replay alone to perform the best. This may be because the WebVision contains highly abstract and noisy classes such as “Spiral” or “Cinema,” making it difficult for Self-Centered filter to sample from correct clusters. Please refer to the appendix for further detail.

**Purification Comparison.** Table 6 compares the purification performance with the state-of-the-art noise detection methods based on the training dynamics, including AUM [67] and INCV [11]. We notice that the performance of AUM and INCV dreadfully declines when detecting label noise among only a small set of data, which is inevitable in online task-free setting, whereas SPR can filter superbly even with a small set of data. Even a non-stochastic version of our Self-Centered filter performs better than the baselines. Encouragingly, our method is further improved by introducing stochastic ensembles.

**Additional Experiments.** The appendix reports more experimental results, including SPR’s noise-free performance, CIFAR-100 filtering performance, episode robustness, purified & delayed buffer size analysis, ablation of stochastic ensemble size, variance analysis, and data efficiency of Self-Replay.

## 6. Conclusion

We presented the Self-Purified Replay (SPR) framework for noisy labeled continual learning. At the heart of our framework is Self-Replay, which leverages self-supervised learning to mitigate forgetting and erroneous noisy label signals. The Self-Centered filter maintains a purified replay buffer via centrality-based stochastic graph ensembles. Experiments on synthetic and real-world noise showed that our framework can maintain a very pure replay buffer even with highly noisy data streams while significantly outperforming many combinations of noisy label learning and continual learning baselines. Our results shed light on using self-supervision to solve the problems of continual learning and noisy labels jointly. Specifically, it would be promising to extend SPR to maintain a not only pure but also more diversified purified buffer.



## 7. Acknowledgement

We express our gratitude for the helpful comments on the manuscript by Junsoo Ha, Soochan Lee and the anonymous reviewers for their thoughtful suggestions. This research was supported by the international cooperation program by NRF of Korea (NRF-2018K2A9A2A11080927), Basic Science Research Program through the National Research Foundation of Korea (NRF) (2020R1A2B5B03095585), and Institute of Information & communications Technology Planning & Evaluation (IITP) grant (No.2019-0-01082, SW StarLab). Gunhee Kim is the corresponding author.

## References

- [1] R. Aljundi. *Continual Learning in Neural Networks*. PhD thesis, Department of Electrical Engineering, KU Leuven, 2019. 6
- [2] R. Aljundi, L. Caccia, E. Belilovsky, M. Caccia, M. Lin, L. Charlin, and T. Tuytelaars. Online continual learning with maximally interfered retrieval. In *NeurIPS*, 2019. 1, 6, 7, 12
- [3] R. Aljundi, R. Marcus, and T. Tuytelaars. Selfless sequential learning. In *ICLR*, 2019. 1, 5, 12
- [4] E. Arazo, D. Ortego, P. Albert, N. E. O'Connor, and K. McGuinness. Unsupervised label noise modeling and loss correction. In *ICML*, 2019. 6, 13
- [5] D. Arpit, S. Jastrzebski, N. Ballas, D. Krueger, E. Bengio, M. S. Kanwal, T. Maharaj, A. Fischer, A. Courville, Y. Bengio, and S. Lacoste-Julien. A closer look at memorization in deep networks. In *ICML*, 2017. 3, 6, 13
- [6] P. Bonacich and P. Lloyd. Eigenvector-like measures of centrality for asymmetric relations. *Social Networks*, 3:191–201, 2001. 4
- [7] M. Caron, I. Misra, J. Marial, P. Goyal, P. Bojanowski, and A. Joulin. Unsupervised learning of visual features by contrasting cluster assignments. In *NeurIPS*, 2020. 3
- [8] R. Caruana. Multitask learning. *Machine Learning*, 28:41–75, 1997. 7, 13, 18
- [9] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny. Efficient lifelong learning with a-gem. In *ICLR*, 2019. 5, 12
- [10] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. Torr, and M. Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486v4*, 2019. 5, 12
- [11] P. Chen, B. Liao, G. Chen, and S. Zhang. Understanding and utilizing deep neural networks trained with noisy labels. In *ICML*, 2019. 6, 7, 8, 13, 14, 15
- [12] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020. 1, 3, 6, 13
- [13] T. Chen, X. Zhai, M. Ritter, M. Lucic, and N. Houlsby. Self-supervised gans via auxiliary rotation loss. In *CVPR*, 2019. 6, 13
- [14] C. d’Auteume, S. Ruder, L. Kong, and D. Yogatama. Episodic memory in lifelong language learning. In *NeurIPS*, 2019. 6, 13
- [15] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society*, 1:1–38, 1991. 4, 12
- [16] J. Devlin, M.W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019. 1
- [17] C. Doersch, A. Gupta, and A. A. Efros. Unsupervised visual representation learning by context prediction. In *ICCV*, 2016. 6, 13
- [18] S. Farquhar and Y. Gal. Towards robust evaluations of continual learning. *arXiv preprint arXiv:1805.09733*, 2019. 6
- [19] Enrico Fini, Stéphane Lathuilière, Enver Sangineto, Moin Nabi, and Elisa Ricci. Online continual learning under extremem memory constraints. In *ECCV*, 2020. 5, 12
- [20] R. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999. 1, 2
- [21] Y. Ge, D. Chen, and H. Li. Mutual mean-teaching: Pseudo label refinery for unsupervised domain adaptation on person re-identification. In *ICLR*, 2020. 6, 13
- [22] S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *ICLR*, 2018. 6, 13
- [23] J. Goldberger and E. Ben-Reuven. Training deep neural networks using a noise adaptation layer. In *ICLR*, 2017. 6, 13
- [24] J.B. Grill, F. Strub, F. Altche, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko. Big self-supervised models are strong semi-supervised learners. In *NeurIPS*, 2020. 1, 3
- [25] G. Gupta, K. Yadav, and L. Paull. La-maml: Look-ahead meta learning for continual learning. In *NeurIPS*, 2020. 6, 13
- [26] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *NeurIPS*, 2018. 6, 13
- [27] J. Han, P. Luo, and X. Wang. Deep self-learning from noisy labels. In *ICCV*, 2019. 6, 13
- [28] H. Harutyunyan, K. Reing, G. V. Steeg, and A. Galstyan. Improving generalization by controlling label-noise information in neural network weights. In *ICML*, 2020. 3, 6, 13
- [29] Tyler L Hayes, Nathan D Cahill, and Christopher Kanan. Memory efficient experience replay for streaming learning. In *2019 International Conference on Robotics and Automation (ICRA)*, 2019. 1, 6, 12
- [30] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020. 1, 3, 6, 13
- [31] D. Hendrycks, M. Mazeika, D. Wilson, and K. Gimpel. Using trusted data to train deep networks on labels corrupted by severe noise. In *NIPS*, 2018. 1, 6, 13
- [32] J. Huang, L. Qu, R. Jia, and B. Zhao. O2u-net: A simple noisy label detection approach for deep neural networks. In *ICCV*, 2019. 6, 13

- [33] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Comput*, 3:79–87, 1991. 4, 12
- [34] K. Javed and M. White. Meta-learning representations for continual learning. In *NeurIPS*, 2019. 5, 12
- [35] L. Jiang, D. Huang, M. Liu, and W. Yang. Beyond synthetic noise: Deep learning on controlled noisy labels. In *ICML*, 2020. 6, 13
- [36] L. jiang, Z. Zhou, T. Leung, L. Li, and L. Fei-Fei. Mentornet: learning data-driven curriculum for very deep neural networks on corrupted labels. In *ICML*, 2018. 1, 6, 13
- [37] D. Kim, J. Jeong, and G. Kim. Imbalanced continual learning with partitioning reservoir sampling. In *ECCV*, 2020. 1, 6, 7, 12, 14, 16, 18
- [38] D. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. In *ICLR*, 2015. 13
- [39] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences*, 2017. 1, 5, 7, 12
- [40] J. Kremer, F. Sha, and C. Igel. Robust active label correction. In *AISTATS*, 2018. 6, 13
- [41] A. Krizhevsky and G. Hinton. Learning multiple layers of features from tiny images. Technical report, Computer Science Department, University of Toronto, 2009. 2, 6, 7, 13, 15
- [42] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient based learning applied to document recognition. In *IEEE*, 1998. 2, 6, 13
- [43] K. Lee, X. He, L. Zhang, and L. Yang. Cleannet: Transfer learning for scalable image classifier training with label noise. In *CVPR*, 2018. 1
- [44] S. Lee, J. Ha, D. Zhang, and G. Kim. A neural dirichlet process mixture model for task-free continual learning. In *ICLR*, 2020. 1, 2, 5, 6, 7, 12, 15
- [45] T. Lesort, A. Gepperth, A. Stoian, and D. Filliat. Marginal replay vs conditional replay for continual learning. In *IJ-CANN*, 2019. 1
- [46] Junnan Li, Richard Socher, and Steven CH Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *ICLR*, 2020. 6
- [47] J. Li, Y. Wong, Q. Zhao, and M. Kankanhalli. Learning to learn from noisy labeled data. In *CVPR*, 2019. 6, 13
- [48] J. Li, C. Xiong, and S. Hoi. Mopro: Webly supervised learning with momentum prototypes. In *ICLR*, 2021. 6, 13
- [49] J. Li, P. Zhou, C. Xiong, R. Socher, and S. C. H. Hoi. Prototypical contrastive learning of unsupervised representations. In *ICLR*, 2020. 6, 13
- [50] Wen Li, Limin Wang, Wei Li, Eirikur Agustsson, and Luc Van Gool. Webvision database: Visual learning and understanding from web data. *arXiv preprint arXiv: 1708.02862*, 2017. 2, 6, 13
- [51] Y. Li, J. Yang, Y. Song, L. Cao, J. Luo, and L. Li. Learning from noisy labels with distillation. In *ICCV*, 2017. 6, 13
- [52] Z. Li and D. Hoiem. Learning without forgetting. In *ECCV*, 2016. 5, 12
- [53] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *NeurIPS*, 2017. 1, 5, 12
- [54] I. Loshchilov and F. Hutter. Sgdr: Stochastic gradient descent with warm restarts. In *ICLR*, 2017. 13
- [55] M. Lukasik, S. Bhojanapalli, A. K. Menon, and S. Kumar. Does label smoothing mitigate label noise? In *ICML*, 2020. 13
- [56] Y. Lyu and I.W. Tsang. Curriculum loss: Robust learning and generalization against label corruption. In *ICLR*, 2020. 2
- [57] X. Ma, Y. Wang, M. E. Houle, S. Zhou, S. M. Erfani, S. Xia, S. Wijewickrema, and J. Bailey. Dimensionality-driven learning with noisy labels. In *ICML*, 2018. 3, 6, 13
- [58] E. Malach and S. Shalev-Shwartz. Decoupling "when to update" from "how to update". In *NeurIPS*, 2017. 6, 13
- [59] D. Mandal, S. Bharadwaj, and S. Biswas. A novel self-supervised re-labeling approach for training with noisy labels. In *WACV*, 2020. 6, 13
- [60] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks. *Psychology of learning and motivation*, 24:109–265, 1989. 1, 2
- [61] D. T. Nguyen, C. K. Mummadi, T. P. N. Ngo, T. H. P. Nguyen, L. Beggel, and T. Brox. Self: Learning to filter noisy labels with self-ensembling. In *ICLR*, 2019. 6, 13
- [62] J. Nieminen. On the centrality in a graph. *Scandinavian Journal of Psychology*, 1:332–336, 1974. 4
- [63] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *ECCV*, 2017. 6, 13
- [64] P. Ostyakov, E. Logacheva, R. Suvorov, V. Aliev, G. Sterkin, O. Khomenko, and S. I. Nikolenko. Label denoising with large ensembles of heterogeneous neural networks. In *ECCV*, 2018. 1
- [65] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. In *CVPR*, 2016. 6, 13
- [66] G. Patrini, A. Rozza, A. Menon, R. Nock, and L. Qu. Making deep neural networks robust to label noise: a loss correction approach. In *CVPR*, 2017. 6, 13
- [67] G. Pleiss, T. Zhang, E. R. Elenberg, and K. Q. Weinberger. Identifying mislabeled data using the area under the margin ranking. In *NIPS*, 2020. 1, 3, 6, 7, 8, 13, 14, 15
- [68] A. Prabhu, P. H.S. Torr, and P. K. Dokania. Gdumb: A simple approach that questions our progress in continual learning. In *ECCV*, 2019. 1, 2, 6, 7, 8, 13, 14, 16, 17, 18, 19
- [69] Aljundi Rahaf, Min Lin, Baptiste Goujaud, and Bengio Yoshua. Gradient based sample selection for online continual learning. In *NeurIPS*, 2019. 1, 2, 6, 12
- [70] R. Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological review*, 97(2):285–308, 1990. 1, 2
- [71] S. Reed, H. Lee, D. Anguelov, C. Szegedy, D. Erhan, and A. Rabinovich. Training deep neural networks on noisy labels with bootstrapping. In *ICLR workshop*, 2015. 13

- [72] M. Ren, W. Zeng, B. Yang, and R. Urtasun. Learning to reweight examples for robust deep learning. In *ICML*, 2018. [1](#), [6](#), [7](#), [8](#), [13](#), [14](#), [18](#), [19](#)
- [73] M. Riemer, I. Cases, R. Ajemian, M. Liu, I. Rish, Y. Tu, and G. Tesauro. Learning to learn without forgetting by maximizing transfer and minimizing interference. In *ICLR*, 2019. [2](#), [5](#), [7](#), [12](#)
- [74] D. Rolnick, A. Ahuja, J. Schwarz, T. P. Lillicrap, and G. Wayne. Experience replay for continual learning. In *NeurIPS*, 2019. [1](#)
- [75] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. [1](#), [5](#), [12](#)
- [76] Y. Shen and S. Sanghavi. Learning with bad training data via iterative trimmed loss minimization. In *ICML*, 2019. [1](#), [6](#)
- [77] H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual learning with deep generative replay. In *NeurIPS*, 2017. [1](#), [5](#), [12](#)
- [78] Gustavo Rodrigues Lacerda Silva, Rafael Ribeiro De Medeiros, Brayan Rene Acevedo Jaimes, Carla Caldeira Takahashi, Douglas Alexandre Gomes Vieira, and Antônio De Pádua Braga. Cuda-based parallelization of power iteration clustering for large datasets. *IEEE Access*, 5:27263–27271, 2017. [14](#)
- [79] H. Song, M. Kim, and J. Lee. Selfie: Refurbishing unclean samples for robust deep learning. In *ICML*, 2019. [6](#), [13](#)
- [80] L. R. Squire. Two forms of human amnesia: an analysis of forgetting. *Journal of Neuroscience*, 6:635–640, 1981. [2](#)
- [81] D. Tanaka, D. Ikami, T. Yamasaki, and K. Aizawa. Joint optimization framework for learning with noisy labels. In *CVPR*, 2018. [6](#), [13](#)
- [82] B. Tang and D. S. Matteson. Graph-based continual learning. In *ICLR*, 2021. [6](#), [13](#)
- [83] S. Thrun. Is learning the n-th thing any easier than learning the first? In *Advances in neural information processing systems*, 1996. [2](#)
- [84] G. M. van de Ven and S. T. Andreas. Three scenarios for continual learning. In *NeurIPS Continual Learning workshop*, 2019. [6](#)
- [85] A. Veit, N. Alldrin, G. Chechik, I. Krasin, A. Gupta, and S. Belongie. Learning from noisy large-scale datasets with minimal supervision. In *CVPR*, 2017. [6](#), [13](#)
- [86] J. S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)*, 11(1):37–57, 1985. [7](#), [16](#), [18](#)
- [87] R. von Mises and H. Pollaczek-Geiringer. Practical methods of solving equations. *Journal of Applied Mathematics and Mechanics*, 9:152–164, 1929. [4](#)
- [88] Y. Wang, W. Liu, X. Ma, J. Bailey, H. Zha, L. Song, and S. Xia. Iterative learning with open-set noisy labels. In *CVPR*, 2018. [6](#), [13](#)
- [89] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, and J. Bailey. Symmetric cross entropy for robust learning with noisy labels. In *ICCV*, 2019. [1](#), [2](#), [6](#), [7](#), [8](#), [14](#), [18](#), [19](#)
- [90] H. Wei, L. Feng, X. Chen, and B. An. Combating noisy labels by agreement: A joint training method with co-regularization. In *CVPR*, 2020. [1](#), [6](#), [7](#), [8](#), [13](#), [14](#), [18](#), [19](#)
- [91] Y. Xu, P. Cao, Y. Kong, and Y. Wang. L<sub>dmi</sub>: An information-theoretic noise-robust loss function. In *NeurIPS*, 2019. [6](#), [13](#)
- [92] M. Ye, X. Zhang, P. C. Yuen, and S. Chang. Unsupervised embedding learning via invariant and spreading instance feature. In *CVPR*, 2019. [6](#), [13](#)
- [93] K. Yi and J. Wu. Probabilistic end-to-end noise correction for learning with noisy labels. In *CVPR*, 2019. [6](#), [7](#), [8](#), [13](#), [14](#), [18](#), [19](#)
- [94] J. Yoon, E. Yang, J. Lee, and S. J. Hwang. Lifelong learning with dynamically expandable networks. In *ICLR*, 2018. [1](#), [5](#), [12](#)
- [95] X. Yu, B. Han, J. Yao, G. Niu, I. Tsang, and M. Sugiyama. How does disagreement help generalization against label corruption? In *ICML*, 2019. [6](#), [13](#)
- [96] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019. [13](#)
- [97] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *ICML*, 2017. [1](#)
- [98] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *ICLR*, 2017. [6](#), [13](#)
- [99] HaiYang Zhang, XiMing Xing, and Liang Liu. Dualgraph: A graph-based method for reasoning about label noise. In *CVPR*, 2021. [6](#)
- [100] S. Zhang and R. Sutton. A deeper look at experience replay. *arXiv preprint arXiv:1712.01275*, 2017. [2](#)
- [101] Yaobin Zhang, Weihong Deng, Mei Wang, Jiani Hu, Xian Li, Dongyue Zhao, and Dongchao Wen. Global-local gc: Large-scale label noise cleansing for face recognition. In *CVPR*, 2020. [6](#)
- [102] Z. Zhang and M. Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. In *NIPS*, 2018. [1](#), [6](#), [13](#)

## Appendix

This appendix enlists the following additional materials.

- I. Posterior in the Beta Mixture Model. Sec. [A](#)
- II. Extended Related Work. Sec. [B](#)
- III. Experiment Details. Sec. [C](#)
- IV. Extended Results & Analyses. Sec. [D](#)
  - i. Efficiency of Eigenvector Centrality. Sec. [D.1](#)
  - ii. Noise-Free Performance. Sec. [D.2](#)
  - iii. Noise Robustness Comparison. Sec. [D.3](#)
  - iv. Features from ImageNet Pretrained Model. Sec. [D.4](#)
  - v. Analyses of Stochastic Ensemble Size ( $E_{max}$ ). Sec. [D.5](#)
  - vi. Filtering performances on CIFAR-100. Sec. [D.6](#)
  - vii. Self-Replay with Noisy Labeled Data. Sec. [D.7](#)
  - viii. Analyses of the Results on WebVision. Sec. [D.8](#)
  - ix. Episode Robustness. Sec. [D.9](#)
  - x. Buffer Size Analysis. Sec. [D.10](#)
  - xi. Variance. Sec. [D.11](#)

### A. Posterior in the Beta Mixture Model

We provide some details about how to fit beta mixture models [33] with the EM-algorithm [15] to obtain the posterior  $p(z|c)$  for the central point with score  $c$ .

In the E-step, fixing  $\pi_z, \alpha_z, \beta_z$ , we update the latent variables using the Bayes rule:

$$\gamma_z(c) = p(z|c) = \frac{\pi_z p(c|\alpha_z, \beta_z)}{\sum_{j=1}^Z \pi_j p(c|\alpha_j, \beta_j)}. \quad (7)$$

In the M-step, fixing the posterior  $\gamma_z(c)$ , we estimate the distribution parameters  $\alpha$  and  $\beta$  using method of moments:

$$\alpha_z = \bar{c}_z \left( \frac{\bar{c}_z(1 - \bar{c}_z)}{s_z^2} - 1 \right), \quad \beta_z = \frac{\alpha_z(1 - \bar{c}_z)}{\bar{c}_z}, \quad (8)$$

where  $\bar{c}_z$  is the weighted average of the centrality scores from all the points in the delayed batch, and  $s_z^2$  is the weighted variance estimate as

$$\bar{c}_z = \frac{\sum_{i=1}^N \gamma_z(c_i) c_i}{\sum_{i=1}^N \gamma_z(c_i)}, \quad (9)$$

$$s_z^2 = \frac{\sum_{i=1}^N \gamma_z(c_i) (c_i - \bar{c}_z)^2}{\sum_{i=1}^N \gamma_z(c_i)}, \quad (10)$$

$$\pi_z = \frac{1}{N} \sum_{i=1}^N \gamma_z(c_i). \quad (11)$$

Finally, we arrive at  $p(z|c) \propto p(z)p(c|z)$ .

## B. Extended Related Work

### B.1. Continual Learning

Continual learning is mainly tackled from three main branches of regularization, expansion, and replay.

**Regularization-based Approaches.** Methods in this branch prevent forgetting by penalizing severe drift of model parameters. Learning without Forgetting [52] employs knowledge distillation to preserve the previously learned knowledge. Similarly, MC-OCL [19] proposes batch-level distillation to balance stability and plasticity in an online manner. Elastic Weight Consolidation [39] finds the critical parameters for each task by applying the Fisher information matrix. Recently, Selfless Sequential Learning [3] enforces representational sparsity, reserving the space for future tasks.

**Expansion-based Approaches.** Many methods in this branch explicitly constrain the learned parameters by freezing the model and instead allocate additional resources to learn new tasks. Progressive Neural Network [75] prevent forgetting by prohibiting any updates on previously learned parameters while allocating new parameters for the training of the future tasks. Dynamically Expandable Networks[94] decides on the number of additional neurons for learning new tasks using  $L2$  regularization for sparse and selective retraining. CN-DPM [44] adopts the Bayesian nonparametric framework to expand the model in an online manner.

**Replay-based Approaches.** The replay-based branch maintains a fixed-sized memory to rehearse back to the model to mitigate forgetting. The fixed-sized memory could be in the form of a buffer for the data samples of previous tasks or the form of generative model weights [77] to generate the previous tasks' data. GEM [53] and AGEM [9] use a buffer to constrain the gradients in order to alleviate forgetting. In [10], training a model even on tiny episodic memory can achieve an impressive performance. Some recent approaches[73, 34] combine rehearsal with meta-learning to find the balance between transfer and interference.

**Online Sequential Learning.** Online sequential learning is closely related to continual learning research, as it assumes that a model can only observe the training samples once before discarding them. Thus, it is a fundamental problem to maintain the buffer or selecting the samples to be rehearsed. ExStream [29] proposes the buffer maintenance method by clustering the data in an online manner. GSS [69] formulates sample selection for the buffer as a constraint reduction, while MIR [2] proposes a sample retrieving method from the buffer by selecting the most interfered samples. Considering real-world data are often imbalanced and multi-labeled, PRS [37] tackles this problem by partitioning the buffer for each class and maintaining it to be balanced. Also, combining graphs or meta-learning with online continual learning has been studied. Graphs are



adopted to represent the relational structures between samples [82], and the meta-loss is applied for learning not only model weights but also per-parameter learning rates [25]. Recently, GDumb [68] and MBPA++ [14] show training a model at inference time improves the overall performance.

## B.2. Noisy Labels

Learning with noisy labeled data has been a long-studied problem. In several works [98, 5, 57] make an important empirical observation that DNNs usually learn the clean data first then subsequently memorize the noisy data. Recently, a new benchmark [35] has been proposed to simulate real-world label noise from the Web. Noisy labeled data learning can be categorized into loss regularization, data re-weighting, label cleaning, clean sample selection via training dynamics.

**Loss Regularization.** This approach designs the noise correction loss so that the optimization objective is equivalent to learning with clean samples. [66] proposes using a noise transition matrix for loss correction. [23] appends a new layer to DNNs to estimate the noise transition matrix while [31] additionally uses a small set of clean data. [102] studies a set of theoretically grounded noise-robust loss functions that can be considered a generalization of the mean absolute error and categorical cross-entropy. [91, 28] propose new losses based on information theory. [47] adopts the meta-loss to find noise-robust parameters. [4] uses a bootstrapping loss based on the estimated noise distribution.

**Data Re-weighting.** This approach suppresses the contribution of noisy samples by re-weighting the loss. [72] utilizes meta-learning to estimate example importance with the help of a small clean data. [88] uses a Siamese network to estimate sample importance in an open-set noisy setting.

**Label Cleaning.** This approach aims at explicitly repairing the labels. [55] shows that using smooth labels is beneficial in noisy labeled data learning. [81, 93] propose to learn the data labels as well as the model parameters. [71, 79] re-label the samples using the model predictions. Additionally, [40] adopts the active learning strategy to choose the samples to be re-labeled. [85, 51, 21] employ multiple models, while [27, 59, 48] utilize prototypes to refine the noisy labels.

**Training Procedures.** Following the observations that clean data and easy patterns are learned prior to noisy data [98, 5, 57], several works propose filtering methods based on model training dynamics. [36] adopts curriculum learning by selecting small loss samples. [90, 21, 26, 95, 58] identify the clean samples using losses or predictions from multiple models and feed them into another model. [32, 67, 61] filter noisy samples based on the accumulated losses or predictions. [11] proposes to fold the training data and filter clean samples by cross-validating those split data.

## B.3. Self-supervised Learning

Self-supervised learning enables the training of a model to utilize its own unlabeled inputs and often shows remarkable performance on downstream tasks. One example of self-supervised learning uses a pretext task, which trains a model by predicting the data’s hidden information. Some examples include patch orderings [17, 63], image inpainting [65], colorization [92], and rotations [22, 13]. Besides designing heuristic tasks for self-supervised learning, some additional works utilize the contrastive loss. [12] proposes a simpler contrastive learning method, which performs representation learning by pulling the randomly transformed samples closer while pushing them apart from the other samples within the batch. [30] formulates contrastive learning as a dictionary look-up and uses the momentum-updated encoder to build a large dictionary. Recently, [49] extends instance-wise contrastive learning to prototypical contrastive learning to encode the semantic structures within the data.

## C. Experiment Details

We present the detailed hyperparameter setting of SPR training as well as the baselines. We resize the images into  $28 \times 28$  for MNIST [42],  $32 \times 32$  for CIFAR-10 [41], and  $84 \times 84$  for WebVision [50]. We set the size of delayed and purified buffer to 300 for MNIST, 500 for CIFAR-10, and 1000 for WebVision on all methods. We use the batch size of self-supervised learning as 300 for MNIST, 500 for CIFAR-10, and 1000 on WebVision. The batch size of supervised learning is fixed to 16 for all experiments. The number of training epochs for the base and expert network are respectively 3000 and 4000 on all datasets, while fine-tuning epochs for the inference network is 50. The NTXent loss [12] uses a temperature of 0.5, and  $E_{max} = 5$  for SPR. We use the Adam optimizer [38] with setting  $\beta_1 = 0.9$ ,  $\beta_2 = 0.999$ ,  $\epsilon = 0.0002$  for self-supervised training of both base and expert network, and  $\epsilon = 0.002$  for supervised fine-tuning.

The hyperparameters for the baselines are as follows.

- Multitask [8]: We perform i.i.d offline training for 50 epochs with uniformly sampled mini-batches.
- Finetune: We run online training through the sequence of tasks.
- GDumb [68]: As an advantage to GDumb, we allow CutMix [96] with  $p = 0.5$  and  $\alpha = 1.0$ . We use the SGDR [54] schedule with  $T_0 = 1$  and  $T_{mult} = 2$ . Since access to a validation data in task-free continual learning is not natural, the number of epochs is set to 100 for MNIST and CIFAR-10 and 500 for WebVision.

- PRS [37]: We set  $\rho = 0$ .
- L2R [72]: We use meta update with  $\alpha = 1$ , and set the number of clean data per class as 100 and the clean update batch size as 100.
- Pencil [93]: We use  $\alpha = 0.4$ ,  $\beta = 0.1$ , stage1 = 70, stage2 = 200,  $\lambda = 600$ .
- SL [89]: We use  $\alpha = 1.0$ ,  $\beta = 1.0$ .
- JoCoR [90]: We set  $\lambda = 0.1$ .
- AUM [67]: We set the learning rate to 0.1, momentum to 0.9, weight decay to 0.0001 with a batch size of 64 for 150 epochs. We apply random crop and random horizontal flip for input augmentation.
- INCV [11]: We set the learning rate to 0.001, weight decay to 0.0001, a batch size 128 with 4 iterations for 200 epochs. We apply random crop and random horizontal flip for input augmentation.

## D. Extended Results & Analyses

We provide more in-depth results and analyses of the experiments in this section.

### D.1. Efficiency of Eigenvector Centrality

The time and space complexity of Eigenvector centrality is  $O(n^2)$ , where  $n$  is the number of data. Our online scenario constraints the size of  $n$  (Delayed buffer size) to be less than 2% of the entire dataset. Also, for  $k$  classes, the complexity reduces to  $O((n/k)^2)$  since the Self-Centered filter computes per class. On Quadro RTX GPU, building the adjacency matrices took less than 0.0003s. On a CPU, Eigenvector centrality computation took 0.4s, 1.3s, 7.1s for buffers of 300, 500, 1K, respectively, which can speed up to 188 by GPU [78].

### D.2. Noise-Free Performance

Table 4 compares our SPR and Self-Replay’s performance against Gdumb’s reported performances on MNIST and CIFAR-10. Interestingly, our Self-Replay performs better than Gdumb, showing great promise in the direction of self-supervised continual learning in general. However, SPR’s performance is below that of Gdumb when completely noise free. We speculate SPR’s mechanics to retain clean samples lead to a tradeoff with precise class feature coverage which seems to be of relative importance in a noise-free setting.

### D.3. Noise Robustness Comparison

Figure 6 contrasts the noise robustness of the strongest and closest baseline Gdumb to Self-Replay under 40% and

|             | MNIST       | CIFAR-10    |
|-------------|-------------|-------------|
| Gdumb [68]  | <b>91.9</b> | 45.8        |
| Self-Replay | 88.9        | <b>47.4</b> |
| SPR         | 85.5        | 44.3        |

Table 4. **Noise Free performances** of Self-Replay and SPR compared with Gdumb [68]’s reported performances. Buffer size is fixed to 500.

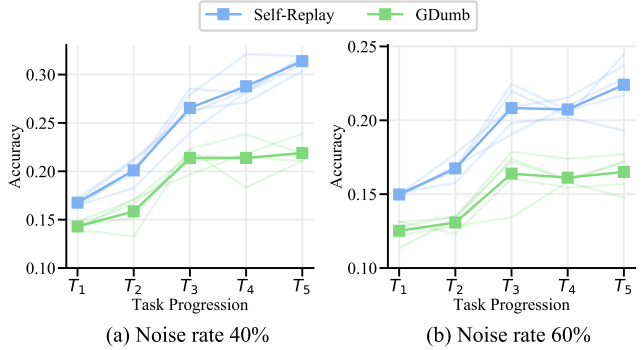


Figure 6. **Noise Robustness** of Self-Replay and Gdumb on CIFAR-10. Both models use conventional reservoir sampling (*i.e.*, uniform random sampling from the input data stream) for the replay (purified) buffer; that is, no purification of the input data is performed. The vivid plots indicate the mean of five random seed experiments.

60% noise levels while removing the Self-Centered filter from our method. Even still, Self-Replay is much more robust against high amounts of noisy labels at every task, validating that Self-Replay alone is able to mitigate the harmful effects of noise to a great extent.

### D.4. Features from ImageNet Pretrained Model

We would like to clarify that our scenario and approach is much different and novel in that, the algorithm assumes an online stream of data and no ground-truth data is available to supervisedly train a noise detector. Not only that, the data we have to work is very small (e.g., 300, 500, 1000) as the purpose is for a Delayed buffer to set aside small amounts from a stream of data for verification by our self-supervisedly trained Expert model. This was also motivated by the empirical evidence that using a supervised learning technique such as AUM [67], INCV [11], and using an ImageNet supervisedly pre-trained model for extracting the features led to worthless performances in the Table 5, 6.

### D.5. Analyses of Stochastic Ensemble Size ( $E_{max}$ )

Figure 7 displays the performance of Stochastic Ensemble by increasing the ensemble sizes ( $E_{max}$ ) from 1 to 40. Stochastic Ensemble performs better in all ensemble sizes than the non-stochastic BMM in terms of the percentages

| noise rate (%)      | CIFAR-10<br>symmetric |      |      |
|---------------------|-----------------------|------|------|
|                     | 20                    | 40   | 60   |
|                     |                       |      |      |
| ImageNet pretrained | -9.0                  | -7.0 | 3.0  |
| Self-supervised     | 75.5                  | 70.5 | 54.3 |

Table 5. **Filtered noisy label percentages** in the purified buffer. We compare filtering performances from the self-supervisedly learned features with the ones from the ImageNet pretrained features. We set  $E_{max} = 5$ .

of filtered noisy labels on both MNIST and CIFAR10 with 60% noisy labels. A substantial boost is seen in the filtering performance up to 10. After 20, the performance starts to plateau on both MNIST and CIFAR-10. The empirically suggested optimal number of  $E_{max}$  may be around 20 for both MNIST and CIFAR-10 and this is further confirmed in Table 6 where we fix  $E_{max} = 20$  and the overall filtering percentage increase by 2.4% on average, compared to the results in the main draft with  $E_{max} = 5$ .

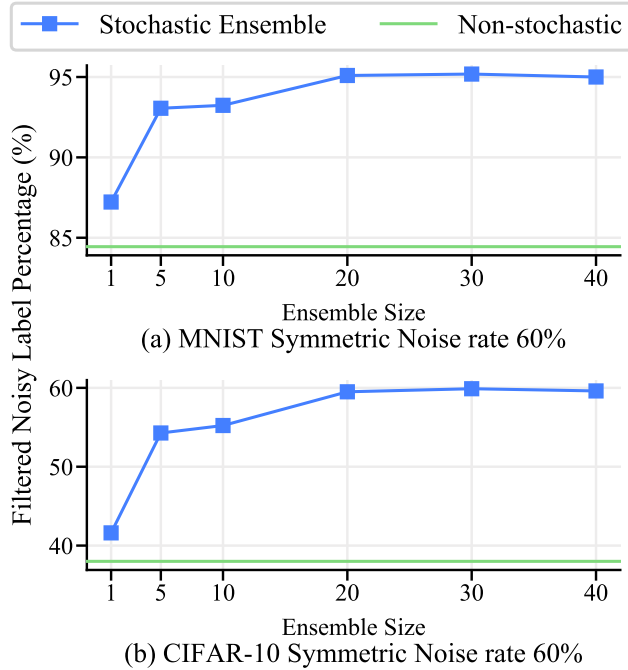


Figure 7. **Filtered noisy label percentages** in the purified buffer by increasing the ensemble size ( $E_{max}$ ) on MNIST and CIFAR-10 with 60% noise rate. Stochastic Ensemble significantly performs better than the static version.

## D.6. Filtering performances on CIFAR-100.

Table 7 compares the filtering performances of SPR with the two state-of-the-art label filtering methods [67, 11] on CIFAR-100. SPR performs the best in all random symmetric noise and superclass symmetric noise with different

| noise rate (%) | MNIST       |             |             |             |             |  | CIFAR-10    |             |             |             |             |  |
|----------------|-------------|-------------|-------------|-------------|-------------|--|-------------|-------------|-------------|-------------|-------------|--|
|                | symmetric   |             |             | asymmetric  |             |  | symmetric   |             |             | asymmetric  |             |  |
|                | 20          | 40          | 60          | 20          | 40          |  | 20          | 40          | 60          | 20          | 40          |  |
| AUM [67]       | 7.0         | 16.0        | 11.7        | 30.0        | 29.5        |  | 36.0        | 24.0        | 11.7        | 46.0        | 30.0        |  |
| INCV [11]      | 23.0        | 22.5        | 14.3        | 37.0        | 31.5        |  | 22.0        | 18.5        | 9.3         | 37.0        | 30.0        |  |
| Non-stochastic | 79.5        | 96.3        | 84.5        | 96.0        | 88.5        |  | 50.5        | 54.5        | 38.0        | 53.0        | 50.5        |  |
| SPR (Ours)     | <b>95.0</b> | <b>96.8</b> | <b>95.0</b> | <b>99.9</b> | <b>97.5</b> |  | <b>79.5</b> | <b>76.3</b> | <b>59.5</b> | <b>72.0</b> | <b>59.0</b> |  |

Table 6. **Filtered noisy label percentages** in the purified buffer. We compare SPR to two other state-of-the-art label filtering methods. We set  $E_{max} = 20$ .

levels of 20%, 40%, and 60%. Even the filtering performance on CIFAR-100 is superior to CIFAR-10. We believe this result is mainly due to the classes in CIFAR100 being more specific than CIFAR10 (e.g., automobile, airplane, bird in CIFAR10 where CIFAR100 has the trees superclass divided into maple, oak, palm, pine, willow), allowing SPR to self-supervisedly learn much more distinct features per class. This result is further reinforced on the WebVision dataset where SPR shows a weakness in filtering abstract classes such as ‘‘Spiral,’’ in which the details can be found in Sec D.8.

| noise rate (%) | random symmetric |             |             | superclass symmetric |             |             |
|----------------|------------------|-------------|-------------|----------------------|-------------|-------------|
|                | 20               | 40          | 60          | 20                   | 40          | 60          |
| AUM [67]       | 33.5             | 46.8        | 13.9        | 25.0                 | 21.4        | 32.4        |
| INCV [11]      | 46.9             | 34.8        | 22.2        | 33.7                 | 27.0        | 15.4        |
| SPR            | <b>82.9</b>      | <b>79.6</b> | <b>64.8</b> | <b>76.5</b>          | <b>69.4</b> | <b>56.0</b> |

Table 7. **Filtered noisy label percentages** in the purified buffer. We compare SPR to two other state-of-the-art label filtering methods on CIFAR-100. We set  $E_{max} = 5$ . The buffer size is set to 5000. ‘‘random symmetric’’ refers to noise randomized across the 100 classes, while ‘‘superclass symmetric’’ refers to noise randomized within the CIFAR-100 superclasses [41, 44].

## D.7. Self-Replay with Noisy Labeled Data

Table 8 compares the overall accuracy of Self-Replay when self-supervised training is performed with and without the delay buffer. Training with the delay buffer means using the samples in both the delay buffer  $B_d$  (red) and the purified buffer  $B_p$  (blue). In contrast, training without the delay buffer means using purified samples  $B_p$  (blue) only. We remind the normalized temperature-scaled cross-entropy loss in the main manuscript as

$$L_{self} = - \sum_{i=1}^{2(B_d+B_p)} \log \frac{e^{u_i^T u_j / \tau}}{\sum_{k=1}^{2(B_d+B_p)} \mathbb{1}_{k \neq i} e^{u_i^T u_k / \tau}}. \quad (12)$$

We observe an approximately 0.6% increase in MNIST and 3.3% increase in CIFAR-10 when using the delay buffer as well, even though it contains noisy labeled samples. We

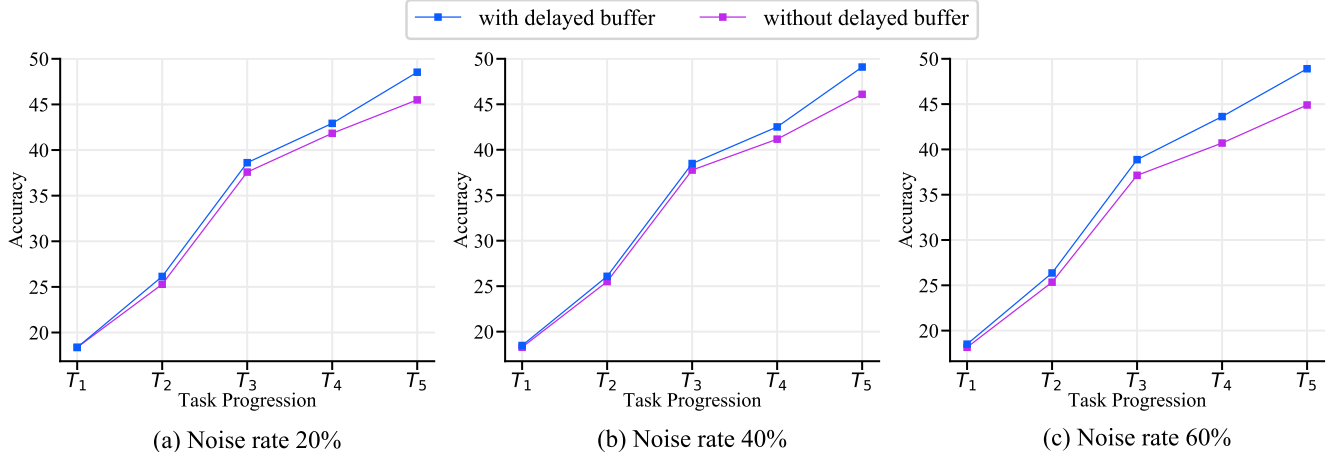


Figure 8. The overall accuracy of SPR over sequential task progression on CIFAR-10 with different noise rates. Training with the delay buffer means that self-supervised learning is performed using the samples in both the delay buffer and the purified buffer, whereas training without the delay buffer means it is done with the samples in the purified buffer only.

speculate that slight improvement is attained in MNIST due to the simplicity of the features. On the other hand, noticeable margins are seen in CIFAR-10, which we further analyze on a per-task basis, shown in Figure 8. The gaps are small in the earlier tasks but become more prominent as more tasks are seen. Moreover, the differences are even more significant when the level of noise rate increases. The take-home message here is that self-supervised training can benefit from the increased data even if it could possibly contain noisy labels.

| noise rate (%) | MNIST<br>symmetric |      |      | CIFAR-10<br>symmetric |      |      |
|----------------|--------------------|------|------|-----------------------|------|------|
|                | 20                 | 40   | 60   | 20                    | 40   | 60   |
| SR with DB     | 91.0               | 91.8 | 91.1 | 48.5                  | 49.1 | 48.9 |
| SR without DB  | 90.3               | 91.0 | 90.5 | 45.5                  | 46.1 | 44.9 |

Table 8. The overall accuracy of SPR with or without the samples in the delay buffer (DB). Self-supervised training can more benefit from more data even though some of them are possibly noisy.

## D.8. Analyses of the Results on WebVision

In the main manuscript, we briefly discuss the observation that Self-Replay and the Self-Centered filter do not synergize well on the WebVision dataset. In this section, we provide extended discussions about this behavior with qualitative and quantitative analyses.

**Qualitative Analysis.** We pointed out that classes such as “Spiral” or “Cinema” are highly abstract by overarching broad related knowledge, which is at the same time corrupted by noise. We show 50 random training data in Figure 10 and Figure 12 for “Spiral” and “Cinema”, respec-

tively. The Self-Centered filter samples for the same classes are also shown in Figure 11 and Figure 13. As visualized, it is not easy to interpret what the central concept is.

This is contrasted by the training samples in the classes “ATM” and “Frog” in Figure 16 and Figure 14. The classes contain noisy samples but represent the class concept without a high amount of abstraction. We also show the Self-Centered filter samples for the classes in Figure 17 and Figure 15. It is much more visually evident what class the samples represent.

**Quantitative Analysis.** Table 9 contrasts the performance of the two topics on GDumb, Self-Replay, Self-Centered filter, and SPR. The Self-Centered filter and SPR use the proposed Self-Centered filtering technique, whereas GDumb and Self-Replay use random sampling instead. The performances also support that random sampling may be a better performer for noisy and abstract classes, as GDumb and Self-Replay attain better performances. On the other hand, for ordinary noisy classes such as “ATM” or “Frog,” the Self-Centered filter and SPR perform stronger than random sampling and show a synergetic effect.

## D.9. Episode Robustness

Table 10 (episode B) and Table 11 (episode C) report the results of two different randomly permuted episodes. We include all of GDumb [68] combinations and the single best performing combination of PRS [37] and CRS [86] for each dataset. Even in two additional random episode experiments, SPR performs much stronger than all the baselines on all datasets with real, symmetric, or asymmetric noise.



|          | GDumb | Self-Replay | Self-Centered filter | SPR  |
|----------|-------|-------------|----------------------|------|
| “Cinema” | 34.3  | 46.4        | 19.6                 | 26.8 |
| “Spiral” | 8.6   | 23.2        | 4.8                  | 9.0  |
| “ATM”    | 23.6  | 52.8        | 26.5                 | 54.0 |
| “Frog”   | 33.0  | 52.4        | 45.2                 | 55.0 |

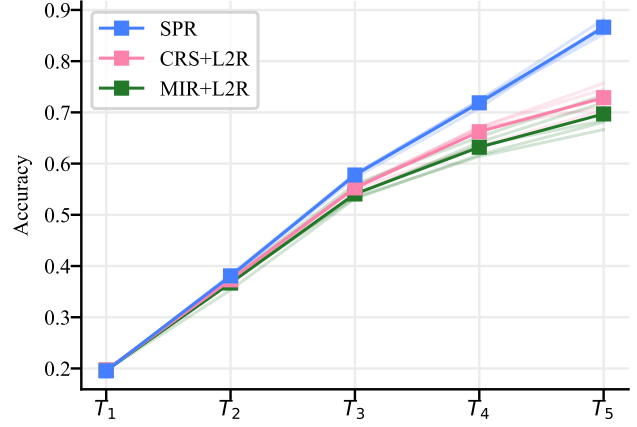
Table 9. Comparison of random sampling based methods (GDumb and Self-Replay) and the methods using the proposed Self-Centered filtering technique (Self-Centered filter and SPR). Random sampling is better for abstract classes such as “Cinema” and “Spiral”, whereas Self-Centered filtering is better for ordinary noisy classes such as “ATM” or “Frog”. The results are the mean of five unique random seed experiments.

### D.10. Buffer Size Analysis

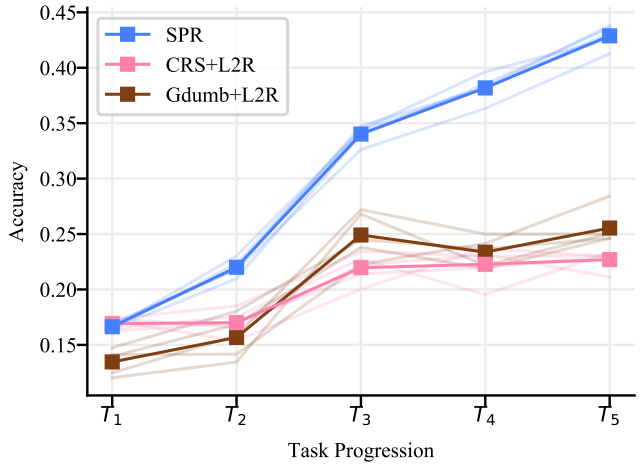
SPR requires a larger amount of memory than some baselines (excluding L2R), but the usage of the memory is different in that, a hold-out memory (Delay Buffer) is used for the purpose of filtering out the noisy labels, while only the Purified Buffer is used to mitigate the amount of forgetting. Hence, simply giving the other baselines a replay buffer twice as big would not be a fair comparison in the viewpoint of continual learning alone. Nonetheless, we run the experiments shown in Table 13, where all of GDumb [68] combinations are allowed twice the buffer size for replay. Even so, SPR using half the buffer size is able to outperform all the other baselines. Furthermore, to inform how the buffer size affects the results, we halve the original used buffer size and report the results in Table 12. SPR still strongly outperforms the baselines in all the datasets and noise rates. These two experiments show that SPR is robust to the buffer size, and its performance is due to self-supervised learning and the clean-buffer management, rather than using the hold-out memory for the Delay buffer.

### D.11. Variance

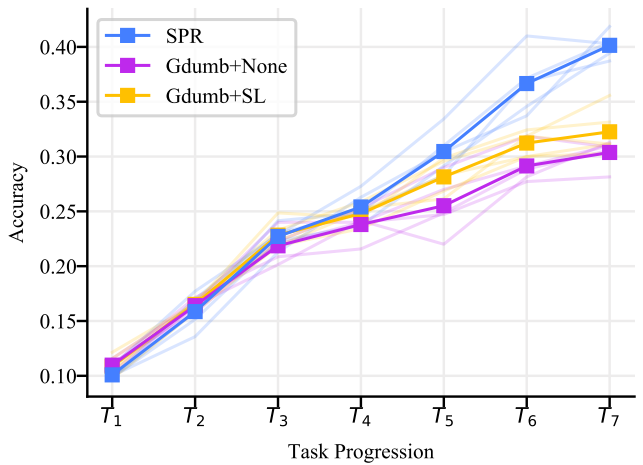
Figure 9 visualizes the variances of top-3 best-performing methods for MNIST, CIFAR-10 with 40% symmetric noise rate, and WebVision with real-noise. Among the symmetric noise experiments with five different random seeds, SPR shows a minor amount of variance throughout the tasks. However, for WebVision, a noticeable amount of fluctuations are seen for all three approaches.



(a) MNIST Symmetric Noise rate 40%



(b) CIFAR-10 Symmetric Noise rate 40%



(c) WebVision

Figure 9. Accuracy and variances of top-3 best-performing methods for MNIST, CIFAR-10 and WebVision.

| noise rate (%)         | MNIST       |             |             |             |             |  | CIFAR-10    |             |             |             |             |  | WebVision             |
|------------------------|-------------|-------------|-------------|-------------|-------------|--|-------------|-------------|-------------|-------------|-------------|--|-----------------------|
|                        | symmetric   |             |             | asymmetric  |             |  | symmetric   |             |             | asymmetric  |             |  | real noise<br>unknown |
|                        | 20          | 40          | 60          | 20          | 40          |  | 20          | 40          | 60          | 20          | 40          |  |                       |
| Multitask 0% noise [8] |             |             | 98.6        |             |             |  |             |             | 84.7        |             |             |  | -                     |
| Multitask [8]          | 94.5        | 90.5        | 79.8        | 93.4        | 81.1        |  | 65.6        | 46.7        | 30.0        | 77.0        | 68.7        |  | 55.5                  |
| CRS + L2R [72]         | 80.8        | 74.1        | 59.7        | 85.3        | 79.8        |  | 29.8        | 23.1        | 16.0        | 36.4        | 36.1        |  | -                     |
| CRS + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 25.1                  |
| PRS + L2R [72]         | 80.7        | 74.0        | 60.4        | 83.2        | 80.1        |  | 30.8        | 22.8        | 15.0        | 36.3        | 32.9        |  | -                     |
| PRS + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 26.5                  |
| MIR + L2R [72]         | 79.6        | 68.6        | 51.6        | 83.2        | 79.5        |  | 31.1        | 21.0        | 14.5        | 34.7        | 33.6        |  | -                     |
| MIR + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 22.6                  |
| GDumb [68]             | 70.1        | 54.6        | 32.3        | 78.2        | 71.1        |  | 29.6        | 22.4        | 16.5        | 33.0        | 30.9        |  | 33.3                  |
| GDumb + L2R [72]       | 67.1        | 59.2        | 40.6        | 70.6        | 68.7        |  | 27.0        | 25.5        | 21.8        | 29.9        | 29.4        |  | -                     |
| GDumb + Pencil [93]    | 70.2        | 53.9        | 35.4        | 77.5        | 70.2        |  | 28.1        | 21.0        | 15.9        | 31.5        | 30.6        |  | 27.5                  |
| GDumb + SL [89]        | 65.6        | 47.5        | 30.5        | 73.3        | 68.5        |  | 27.1        | 22.6        | 16.8        | 33.2        | 31.4        |  | 32.5                  |
| GDumb + JoCoR [90]     | 68.3        | 56.0        | 41.0        | 78.5        | 70.9        |  | 26.6        | 21.1        | 15.9        | 32.9        | 32.2        |  | 22.9                  |
| SPR                    | <b>86.8</b> | <b>87.2</b> | <b>82.1</b> | <b>86.6</b> | <b>85.5</b> |  | <b>42.0</b> | <b>42.4</b> | <b>39.1</b> | <b>44.4</b> | <b>43.3</b> |  | <b>41.6</b>           |

Table 10. **Overall accuracy on episode B** after all sequences of tasks are trained. The buffer size is set to 300, 500, 1000 for MNIST, CIFAR-10, and WebVision, respectively. We report all of GDumb [68] combinations and single best performing combination of PRS [37] and CRS [86]. Some empty slots on WebVision are due to the unavailability of clean samples required by L2R for training [72]. The results are the mean of five unique random seed experiments.

| noise rate (%)         | MNIST       |             |             |             |             |  | CIFAR-10    |             |             |             |             |  | WebVision             |
|------------------------|-------------|-------------|-------------|-------------|-------------|--|-------------|-------------|-------------|-------------|-------------|--|-----------------------|
|                        | symmetric   |             |             | asymmetric  |             |  | symmetric   |             |             | asymmetric  |             |  | real noise<br>unknown |
|                        | 20          | 40          | 60          | 20          | 40          |  | 20          | 40          | 60          | 20          | 40          |  |                       |
| Multitask 0% noise [8] |             |             | 98.6        |             |             |  |             |             | 84.7        |             |             |  | -                     |
| Multitask [8]          | 94.5        | 90.5        | 79.8        | 93.4        | 81.1        |  | 65.6        | 46.7        | 30.0        | 77.0        | 68.7        |  | 55.5                  |
| CRS + L2R [72]         | 79.9        | 74.9        | 58.2        | 84.4        | 79.4        |  | 29.3        | 24.4        | 16.8        | 37.2        | 37.5        |  | -                     |
| CRS + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 29.9                  |
| PRS + L2R [72]         | 80.5        | 72.3        | 55.2        | 83.8        | 80.1        |  | 30.6        | 23.3        | 16.3        | 37.2        | 36.1        |  | -                     |
| PRS + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 28.5                  |
| MIR + L2R [72]         | 80.3        | 69.7        | 47.1        | 83.0        | 77.6        |  | 28.2        | 21.3        | 15.6        | 36.3        | 34.3        |  | -                     |
| MIR + Pencil [93]      | -           | -           | -           | -           | -           |  | -           | -           | -           | -           | -           |  | 22.4                  |
| GDumb [68]             | 71.8        | 52.8        | 37.5        | 79.2        | 72.1        |  | 28.7        | 23.0        | 16.3        | 34.2        | 31.9        |  | 31.6                  |
| GDumb + L2R [72]       | 67.7        | 58.2        | 42.7        | 69.3        | 67.6        |  | 28.9        | 24.8        | 19.7        | 31.8        | 29.4        |  | -                     |
| GDumb + Pencil [93]    | 69.0        | 54.2        | 37.8        | 78.6        | 71.2        |  | 27.5        | 21.0        | 16.6        | 31.3        | 31.8        |  | 28.5                  |
| GDumb + SL [89]        | 65.4        | 48.4        | 29.1        | 72.4        | 67.7        |  | 28.3        | 22.9        | 15.0        | 31.4        | 31.9        |  | 31.6                  |
| GDumb + JoCoR [90]     | 70.4        | 59.0        | 40.6        | 77.4        | 70.6        |  | 27.8        | 22.3        | 15.5        | 33.4        | 31.7        |  | 24.3                  |
| SPR                    | <b>86.6</b> | <b>87.5</b> | <b>84.4</b> | <b>87.0</b> | <b>87.3</b> |  | <b>43.7</b> | <b>43.1</b> | <b>39.8</b> | <b>44.3</b> | <b>43.2</b> |  | <b>40.2</b>           |

Table 11. **Overall accuracy on episode C** after all sequences of tasks are trained. The buffer size is set to 300, 500, 1000 for MNIST, CIFAR-10, and WebVision, respectively. We report all of GDumb [68] combinations and single best performing combination of PRS [37] and CRS [86]. Some empty slots on WebVision are due to the unavailability of clean samples required by L2R for training [72]. The results are the mean of five unique random seed experiments.

| Buffer size<br>noise rate (%) | MNIST     |      |      |            |      |      | CIFAR-10  |      |      |            |         | WebVision  |
|-------------------------------|-----------|------|------|------------|------|------|-----------|------|------|------------|---------|------------|
|                               | symmetric |      |      | asymmetric |      |      | symmetric |      |      | asymmetric |         | real noise |
|                               | 150       |      |      | 150        |      |      | 250       |      |      | 250        |         | 500        |
|                               | 20        | 40   | 60   | 20         | 40   | 20   | 40        | 60   | 20   | 40         | unknown |            |
| GDumb + L2R [72]              | 64.8      | 55.5 | 37.8 | 71.2       | 66.8 | 23.2 | 22.1      | 19.3 | 28.4 | 24.8       | -       |            |
| GDumb + Pencil [93]           | 59.3      | 48.1 | 36.4 | 76.4       | 66.6 | 25.6 | 17.9      | 13.9 | 27.6 | 26.8       | 21.1    |            |
| GDumb + SL [89]               | 61.5      | 41.3 | 31.1 | 66.8       | 56.8 | 20.7 | 19.8      | 18.8 | 29.2 | 26.4       | 26.4    |            |
| GDumb + JoCoR [90]            | 66.8      | 60.9 | 33.0 | 74.4       | 66.3 | 23.8 | 18.9      | 14.2 | 26.2 | 26.2       | 23.0    |            |
| SPR                           | 82.6      | 85.4 | 81.2 | 77.0       | 81.6 | 41.2 | 41.2      | 37.8 | 42.8 | 41.3       | 39.4    |            |

Table 12. **Overall accuracy on the half buffer size** after all sequences of tasks are trained. The buffer size is set to 150, 250, 500 for MNIST, CIFAR-10, and WebVision, respectively. We report all of GDumb [68] combinations. An empty slot on WebVision are due to the unavailability of clean samples required by L2R for training [72].

| MNIST                         |                  |      |      |                   |      | CIFAR-10          |      |      |                    |      | WebVision          |
|-------------------------------|------------------|------|------|-------------------|------|-------------------|------|------|--------------------|------|--------------------|
| Buffer size<br>noise rate (%) | symmetric<br>600 |      |      | asymmetric<br>600 |      | symmetric<br>1000 |      |      | asymmetric<br>1000 |      | real noise<br>2000 |
|                               | 20               | 40   | 60   | 20                | 40   | 20                | 40   | 60   | 20                 | 40   | unknown            |
| GDumb + L2R [72]              | 76.7             | 62.6 | 51.9 | 79.7              | 73.3 | 31.4              | 27.3 | 24.0 | 35.0               | 36.0 | -                  |
| GDumb + Pencil [93]           | 72.1             | 58.5 | 39.4 | 75.3              | 73.5 | 31.2              | 24.5 | 16.4 | 38.6               | 35.5 | 33.0               |
| GDumb + SL [89]               | 66.0             | 47.2 | 31.7 | 79.0              | 74.8 | 33.1              | 23.2 | 17.7 | 40.4               | 37.3 | 38.5               |
| GDumb + JoCoR [90]            | 74.3             | 57.8 | 42.5 | 78.3              | 76.0 | 31.9              | 22.8 | 17.4 | 42.5               | 38.1 | 27.0               |

Table 13. **Overall accuracy on the double buffer size for all of GDumb combinations** after all sequences of tasks are trained. The buffer size is set to 600, 1000, 2000 for MNIST, CIFAR-10, and WebVision, respectively. An empty slot on WebVision are due to the unavailability of clean samples required by L2R for training [72]. Note that SPR outperforms all of GDumb [68] combinations with the buffer size of 300, 500, 1000 for MNIST, CIFAR-10, and WebVision, respectively.

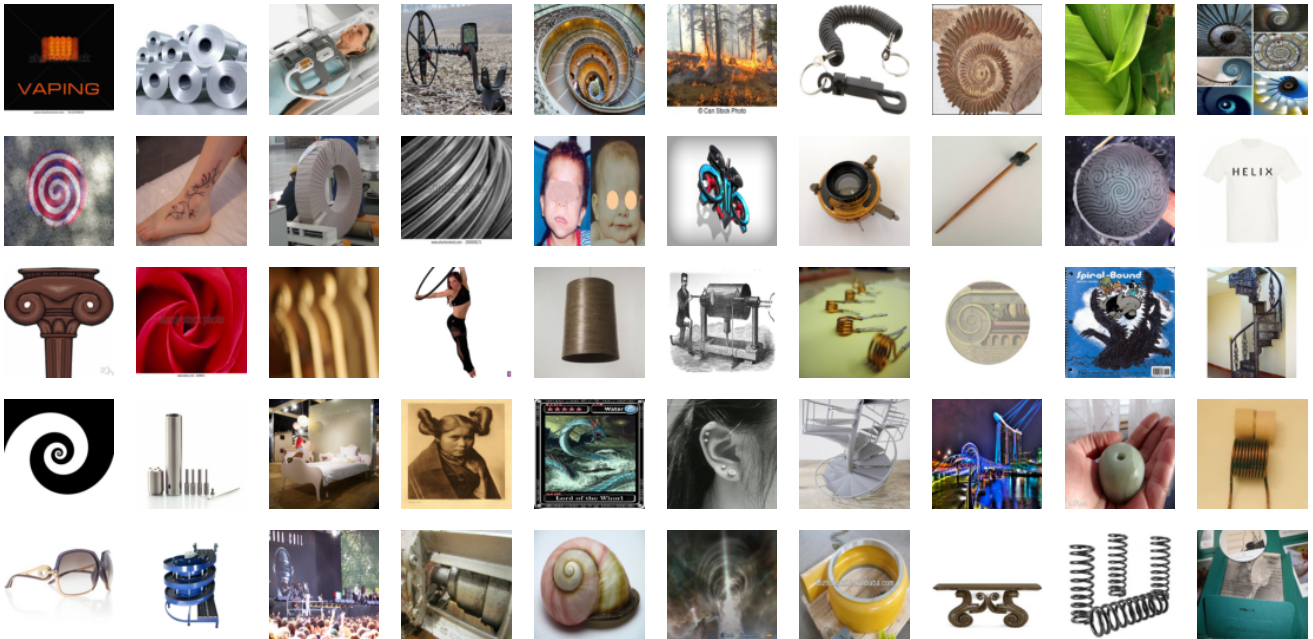


Figure 10. 50 random samples of the “Spiral” class from the training set.

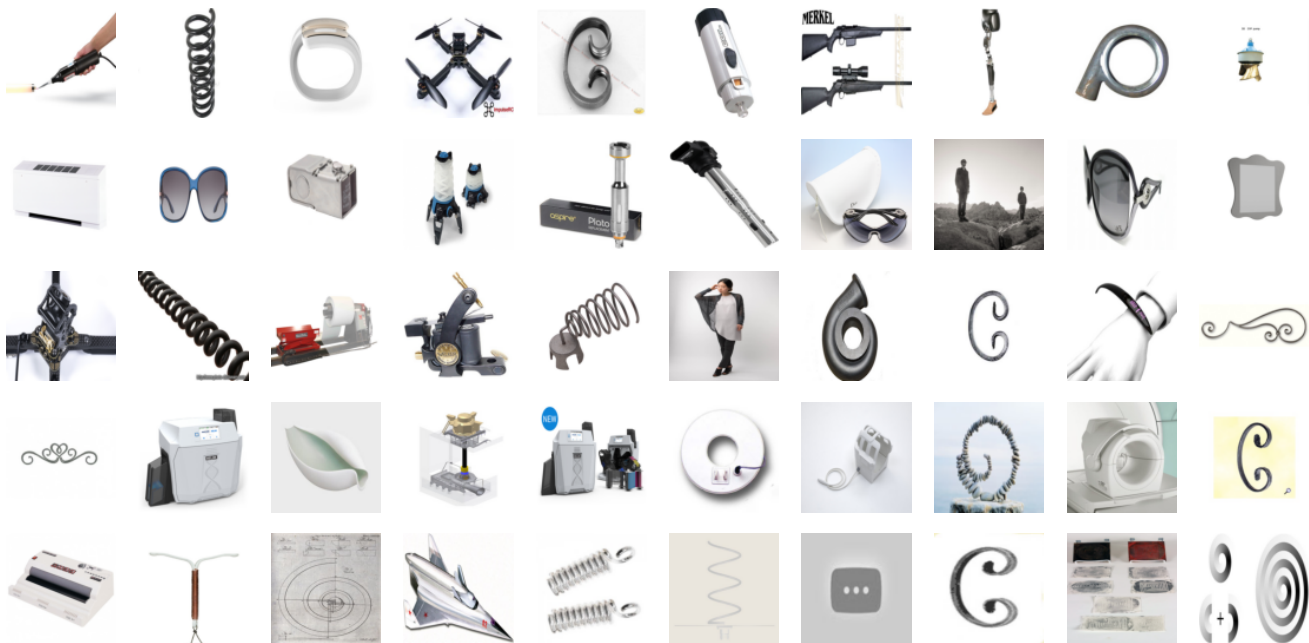


Figure 11. 50 random training samples of the “Spiral” class from the purified buffer.



Figure 12. 50 random samples of the “Cinema” class from the training set.



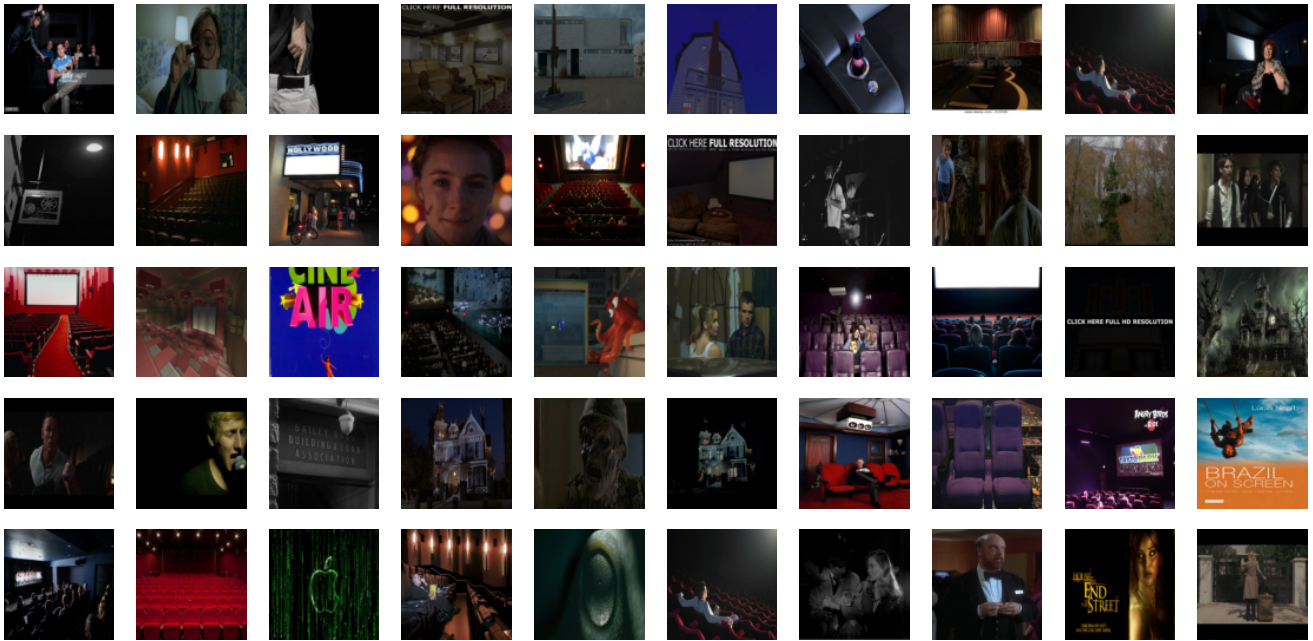


Figure 13. 50 random training samples of the “Cinema” class from the purified buffer.

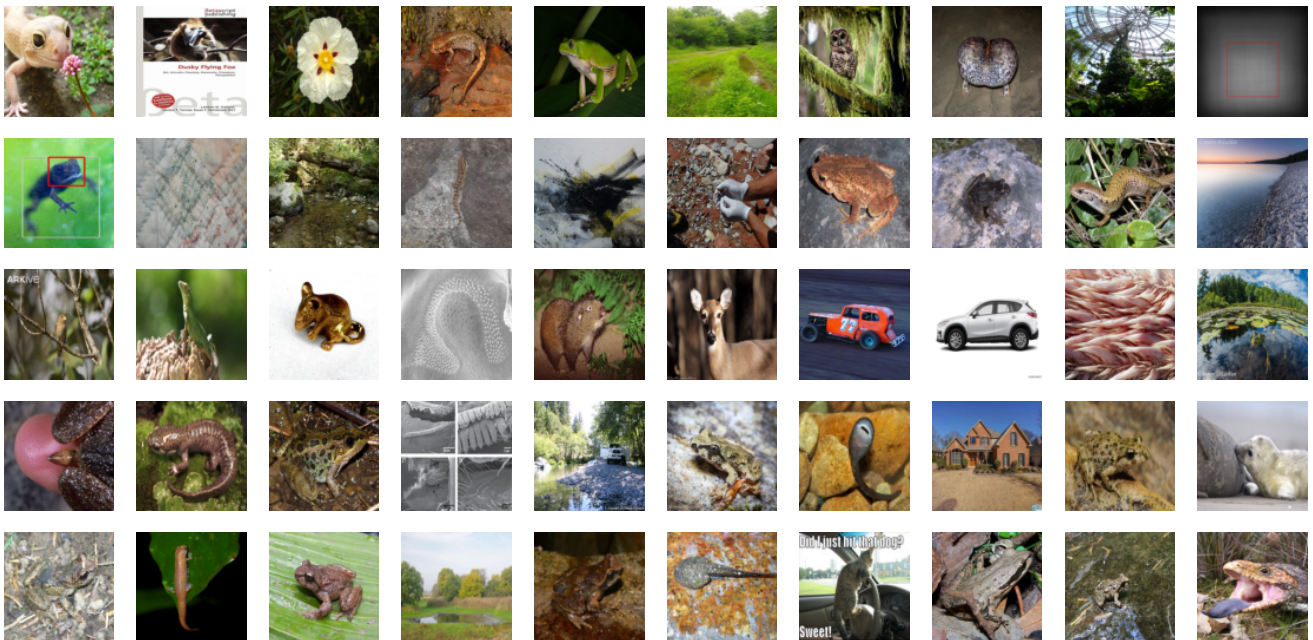


Figure 14. 50 samples of the “Frog” class from the training set.



Figure 15. 50 training samples of the “Frog” class from the purified buffer.



Figure 16. 50 samples of the “ATM” class from the training set





Figure 17. 50 random training samples of the “ATM” class from the purified buffer.